

SKADA Advanced Practitioner – Time Series

객관식 문제 해설

Model Evaluation: Train, Validation, and Test

- Q1) 모델 학습 과정에서 성능을 평가하기 위해 Validation을 수행한다. 다음 중 Validation 과정에 대한 설명으로 옳지 않은 것은?
 - Validation 세트의 목적 중 하나는 하이퍼파라미터를 조정하거나 모델 구조를 선택하는 데 도움을 주는 것이다.
 - Validation 단계를 통해 모델을 개선한 후에는, Training 세트와 Validation 세트를 합쳐 모델을 다시 학습하기도 한다.
 - K-fold cross-validation은 전체 데이터를 훈련과 검증에 고르게 활용할 수 있는 방법이다.
 - Validation 성능이 높을수록 모델의 실제 데이터에 대한 성능도 비례해서 높아진다.

데이터 분할

- 데이터셋이 주어졌을 때 데이터를 적절히 분할하는 것이 필요함
 - 일반적으로 **학습 데이터**, **검증 데이터**, **테스트 데이터**로 분할
- 예) 1,000개의 데이터 중 700개는 학습, 100개는 검증, 200개는 테스트 데이터로 사용
- **학습 데이터**는 모델을 학습하는 용도로 사용
- **검증 데이터**는 학습 과정에서 모델의 성능을 평가하고 하이퍼파라미터 선택을 위해 사용
 - 모델 학습에 직접적으로 사용하지는 않지만 최종 모델 선택에 관여함
- **테스트 데이터**는 학습 및 검증이 완료된 모델을 최종 평가에 사용
 - 학습 및 검증 과정에서는 절대 테스트 데이터를 사용하면 안됨
 - 처음보는 데이터셋으로 평가를 진행하여야 모델의 일반화 성능을 정확히 평가할 수 있음

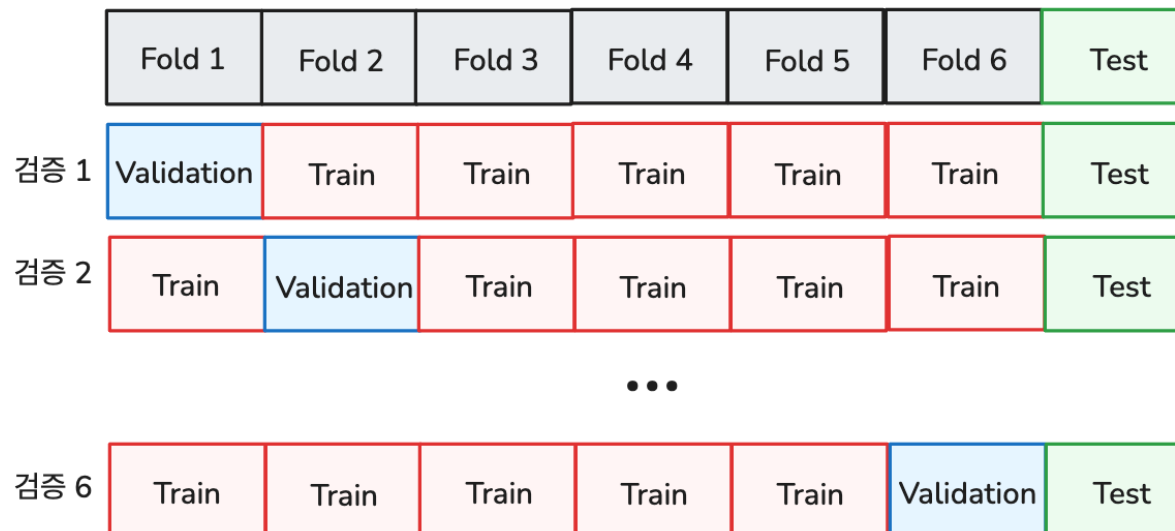
Cross-Validation

- 학습 데이터와 검증 데이터 조합이 하나만 있으면 테스트 데이터에 대한 일반화 성능이 떨어질 수 있음
- 학습 데이터와 검증 데이터의 조합을 여러 번 바꿔가며 평균적인 검증 성능을 측정하는 방식

기본적인
데이터 분할



6-Fold
Cross-Validation



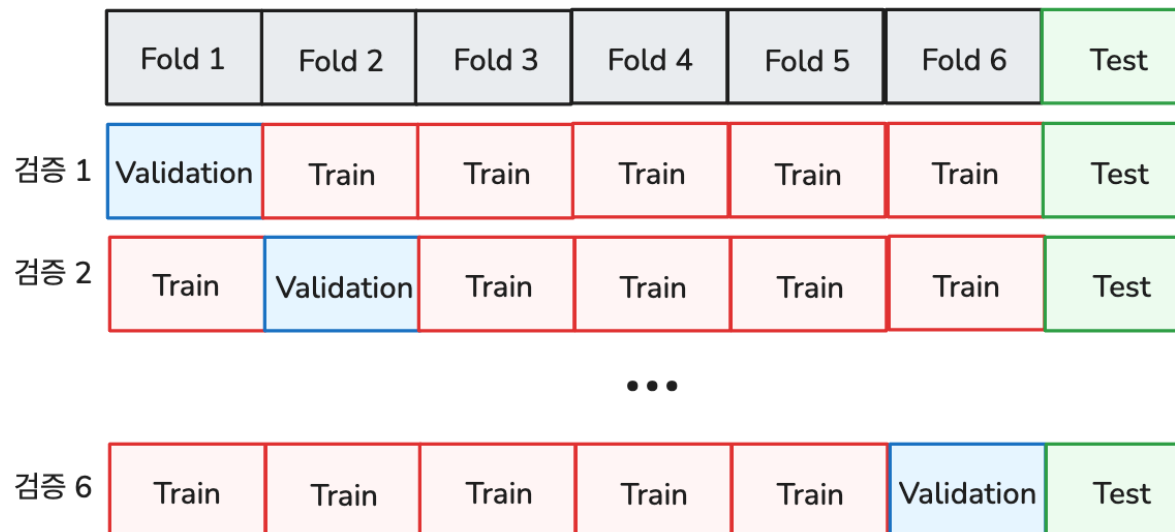
Cross-Validation

- 기본적인 데이터 분할 방식에서는 Validation 결과가 하나
 - 최종 검증 성능 = Validation 결과 하나
- 6-Fold Cross-Validation 에서는 Validation 결과가 여섯
 - 최종 검증 성능 = 여섯 개의 Validation 결과의 평균
 - 더 신뢰할 수 있는 검증 결과로 볼 수 있음

기본적인
데이터 분할

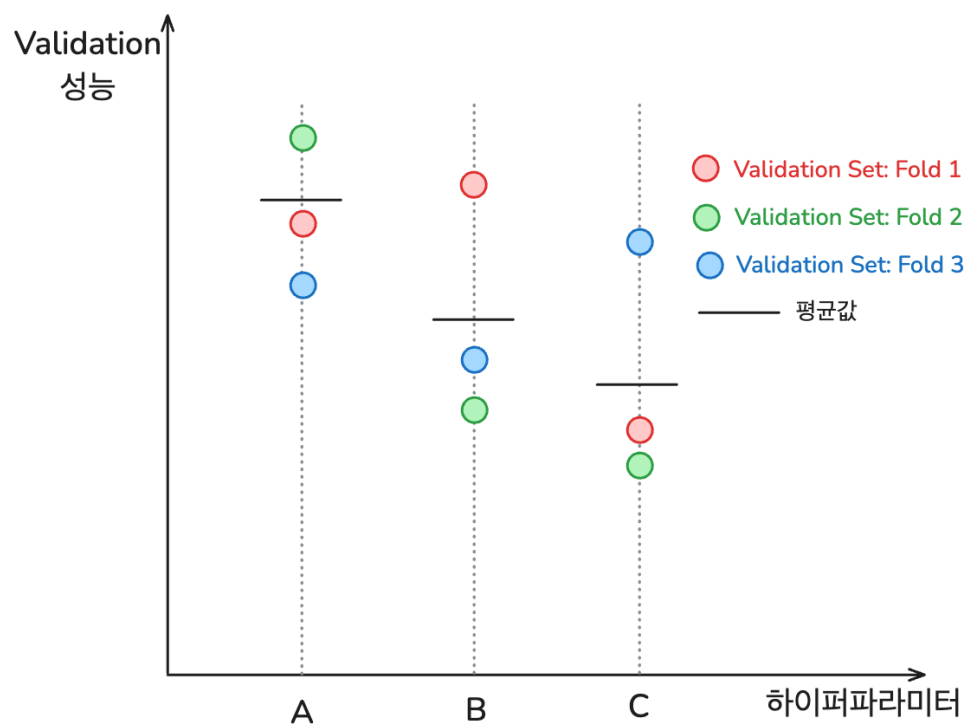


6-Fold
Cross-Validation



Cross-Validation을 이용한 하이퍼파라미터 튜닝

- Cross-Validation을 사용하면 더 정확한 하이퍼파라미터 튜닝을 할 수 있음
 - 만약 한 번의 학습 / 검증 데이터셋 조합으로 튜닝을 했다면 그 선택에 따라 A,B,C 중 최적의 하이퍼파라미터가 모두 다름
 - 여러 번의 학습 / 검증 데이터셋 조합을 통해 평균을 확인하여 최적의 선택 가능



Model Evaluation: Train, Validation, and Test

- Q1) 모델 학습 과정에서 성능을 평가하기 위해 Validation을 수행한다. 다음 중 Validation 과정에 대한 설명으로 옳지 않은 것은?
 - Validation 세트의 목적 중 하나는 하이퍼파라미터를 조정하거나 모델 구조를 선택하는 데 도움을 주는 것이다.
 - Validation 단계를 통해 모델을 개선한 후에는, Training 세트와 Validation 세트를 합쳐 모델을 다시 학습하기도 한다.
 - K-fold cross-validation은 전체 데이터를 훈련과 검증에 고르게 활용할 수 있는 방법이다.
 - Validation 성능이 높을수록 모델의 실제 데이터에 대한 성능도 비례해서 높아진다.
 - Test set에서의 성능 향상은 보장되지 않음. Validation set에 오히려 오버피팅 될 수 있음.

Basic NLP Components: RNN, LSTM, and GRU

- Q2) 다음 RNN과 LSTM, GRU 모듈에 관한 설명 중 옳지 않은 것은?
 - RNN은 시점별로 각 시점의 은닉 상태(hidden state)를 다음 시점으로 전달하기 때문에, 이전 시점들의 문맥 정보를 활용할 수 있다.
 - LSTM의 셀 상태(cell state)는 기존 RNN의 은닉 상태와 다르게, 시퀀스가 길어질 때 정보 흐름이 쉽게 소실되지 않도록 설계되었다.
 - RNN을 개선한 LSTM에서는 RNN과 달리 'Vanishing Gradient' 문제가 발생하지 않는다.
 - GRU는 입력 게이트와 망각 게이트를 하나의 업데이트 게이트로 통합하여, LSTM 대비 파라미터 수가 조금 더 적다.

Recurrent Neural Network (RNN)

가변적인 길이의 Sequence 자료를 입력으로 받아 출력

- 매 Time step 마다 반복적으로 수식을 적용하여 Vector x_t 를 처리

$$\text{New state } h_t = f_{\theta}(\text{Old state } h_{t-1}, x_t)$$

Parameter θ 를 가지는 어떤 함수

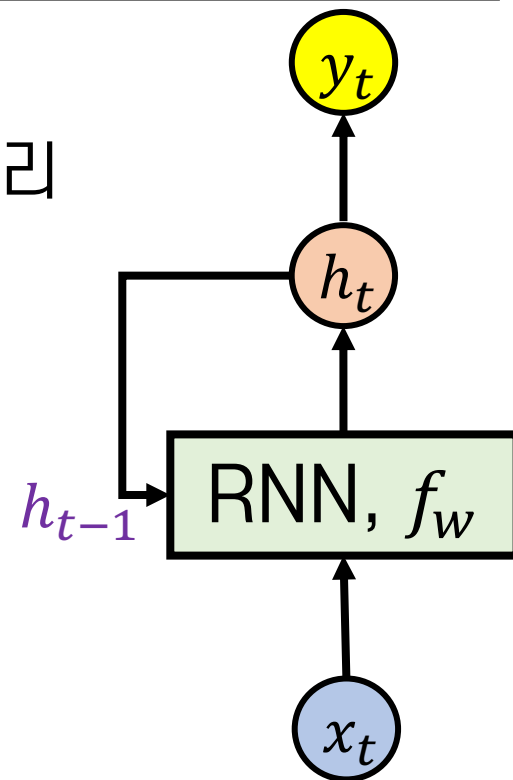
Time step t 에서
의 입력 Vector

e.g., $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$

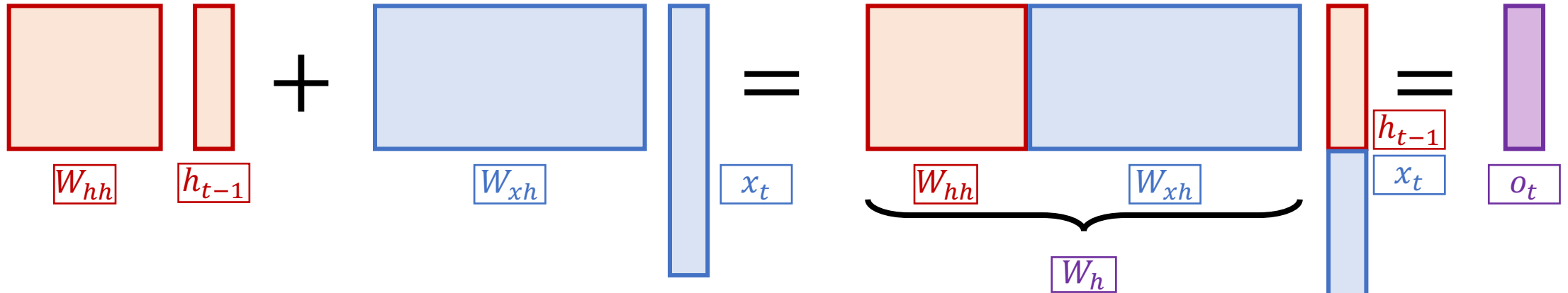
- Activation function으로 주로 Zero-centered인 Tanh를 사용
- 추가로, Target을 예측하기 위하여 h_t 를 사용해 y_t 출력할 수 있음

e.g., $y_t = W_{hy}h_t$

주의: 모든 Time step에서 같은 함수 f_{θ} 와 같은 Parameter θ 적용



RNN의 Hidden-State Vector 계산

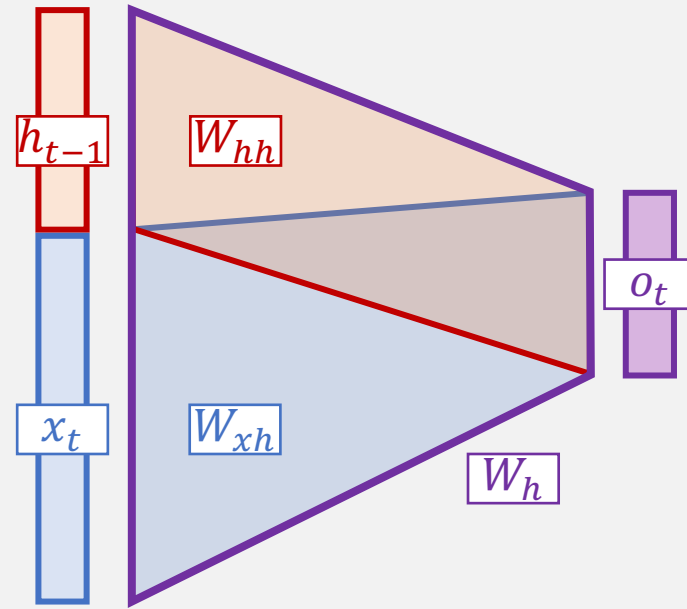


RNN 모델의 기본 구조

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

- h_{t-1}, x_t 를 이어 붙인 후 선형 변환 수행
- 이러한 연산을 Concatenation이라고 지칭
- 세미콜론 ; 으로도 표현함

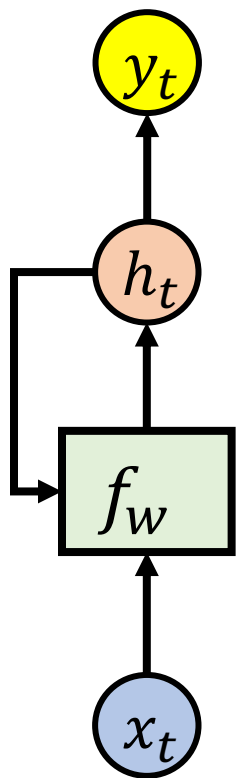
$$h_t = \tanh(W_h[h_{t-1}; x_t])$$



Rolled / Unrolled RNN

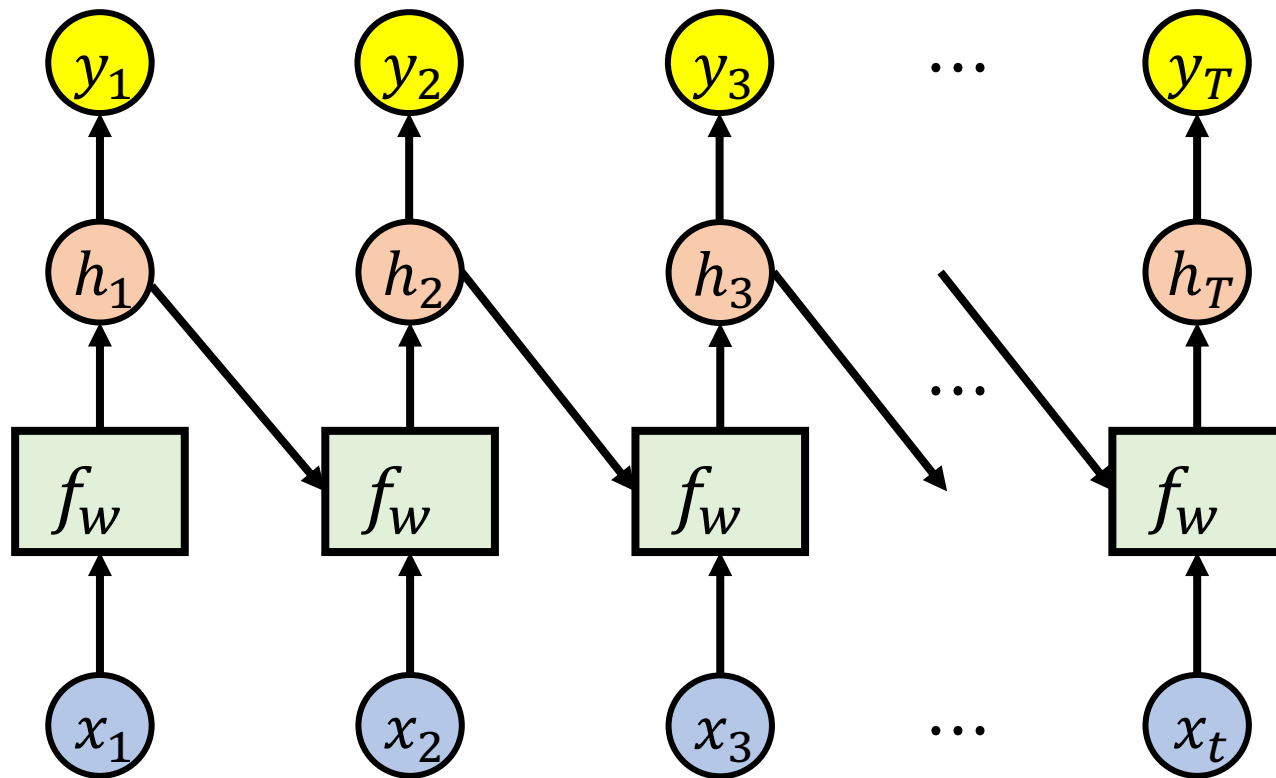
RNN의 기본적인 구조

Rolled RNN



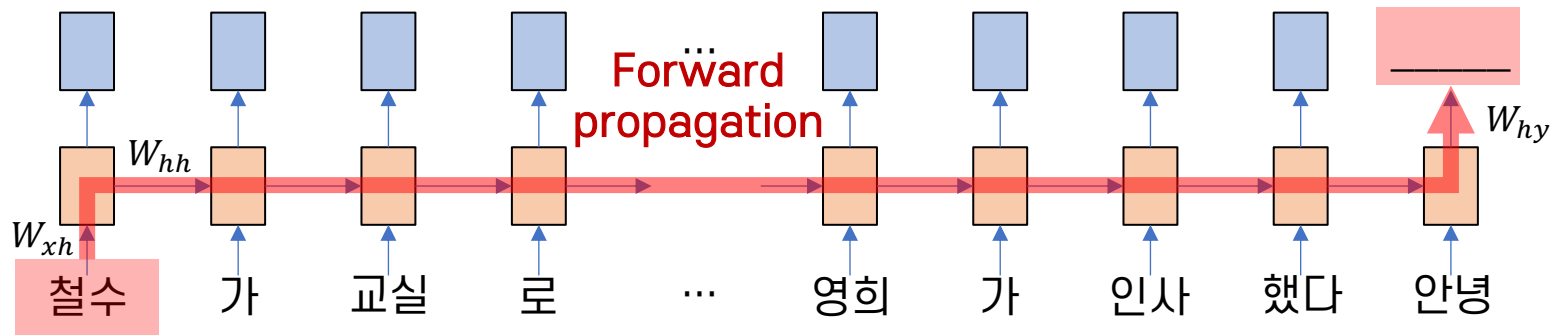
=

Unrolled RNN



$$h_t = f_w(h_{t-1}, x_t) \rightarrow \begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ y_t &= W_{hy}h_t \end{aligned}$$

RNN의 문제점



Forward propagation process

- 1) $h_1 = \tanh(W_{xh}x_1 + W_{hh}h_0 + c)$
- 2) $h_2 = \tanh(W_{xh}x_2 + W_{hh}h_1 + c)$
- 3) ...
- 4) $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + c)$
- 5) $y_t = W_{hy}h_t$

Forward propagation 과정 중 ...

- 1) “철수”라는 정보가 W_{xh} 로 Encoding되어 Hidden-state vector에 저장됨
- 2) Time step이 진행되면서 저장된 다양한 정보를 W_{hh} 를 통해 Hidden-state vector에서 처리
- 3) 마지막 Time step의 Hidden-state vector를 가지고, W_{hy} 를 통해 “철수”라는 Categorical variable로 예측

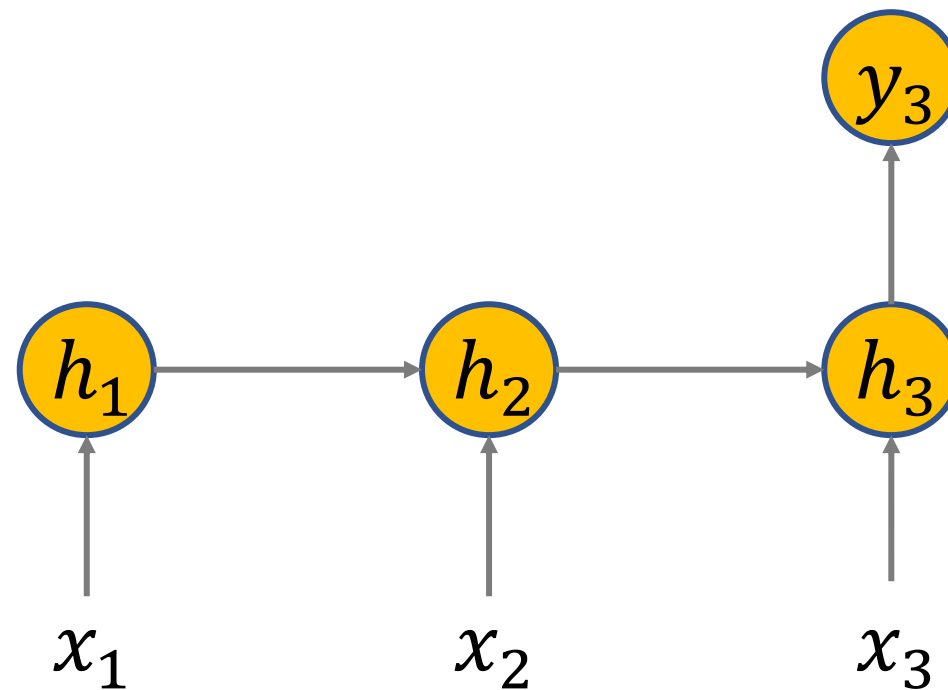
문제점)

- “철수”라는 정보 외의 많은 정보를 고정된 크기의 Hidden state에 저장
- Forward propagation 과정 중에 W_{hh} 를 계속 곱함 ← 이번에 다룰 주제

간단한 RNN 학습 예시

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$



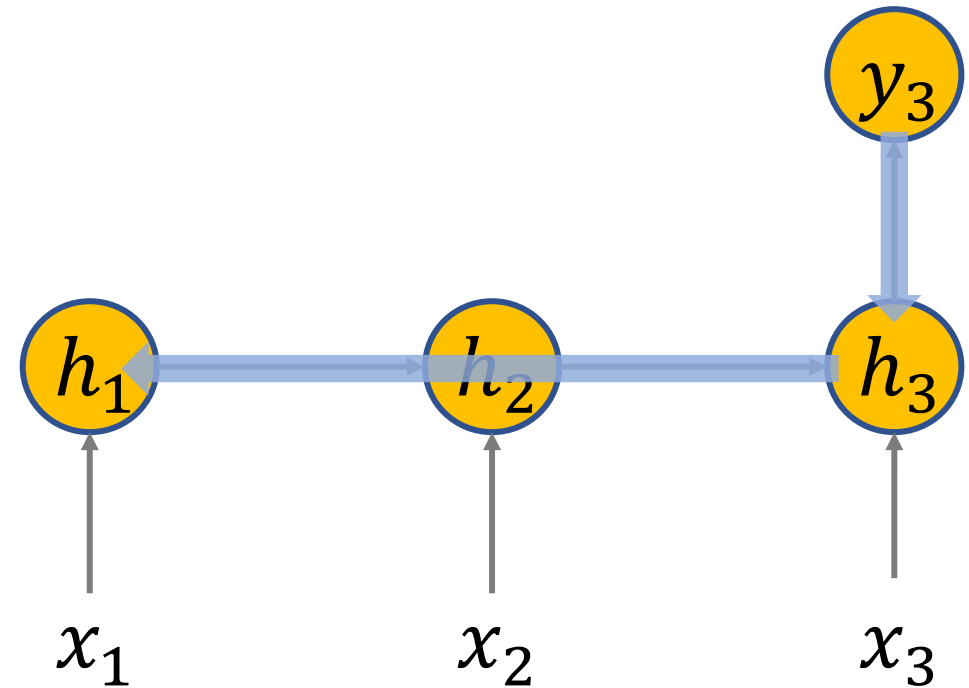
간단한 RNN 학습 예시

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$

Backpropagation

- Let $f = y_3$
- $\frac{df}{dh_1} = ?$



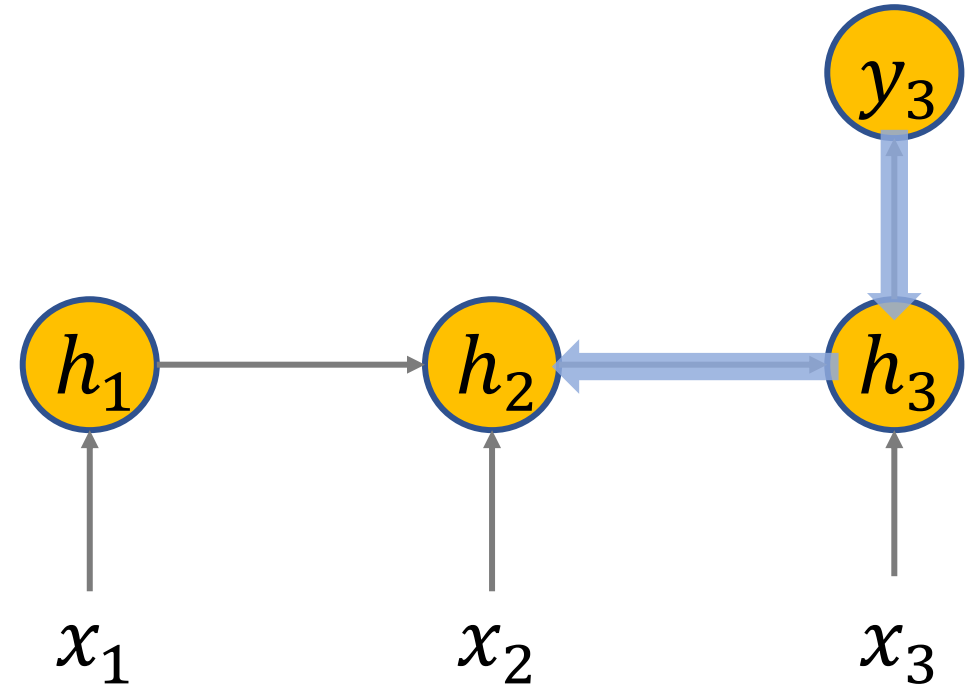
간단한 RNN 학습 예시

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$

Backpropagation

- Let $f = y_3$
- $\frac{df}{dh_1} = \frac{dh_2}{dh_1} \times \frac{df}{dh_2}$



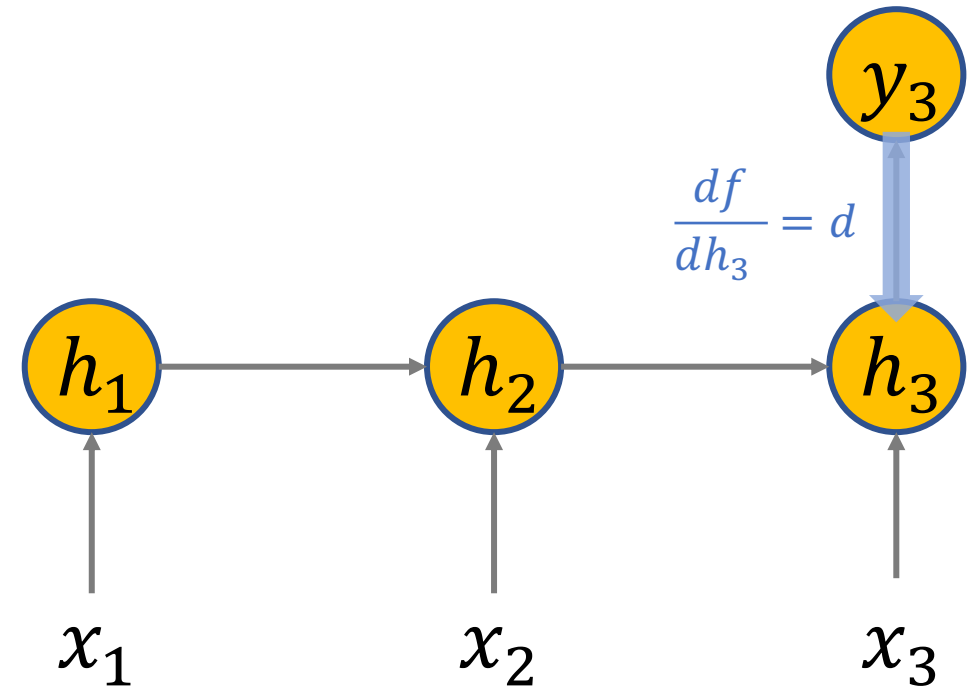
간단한 RNN 학습 예시

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$

Backpropagation

- Let $f = y_3$
- $\frac{df}{dh_1} = \frac{dh_2}{dh_1} \times \frac{dh_3}{dh_2} \times \frac{df}{dh_3}$



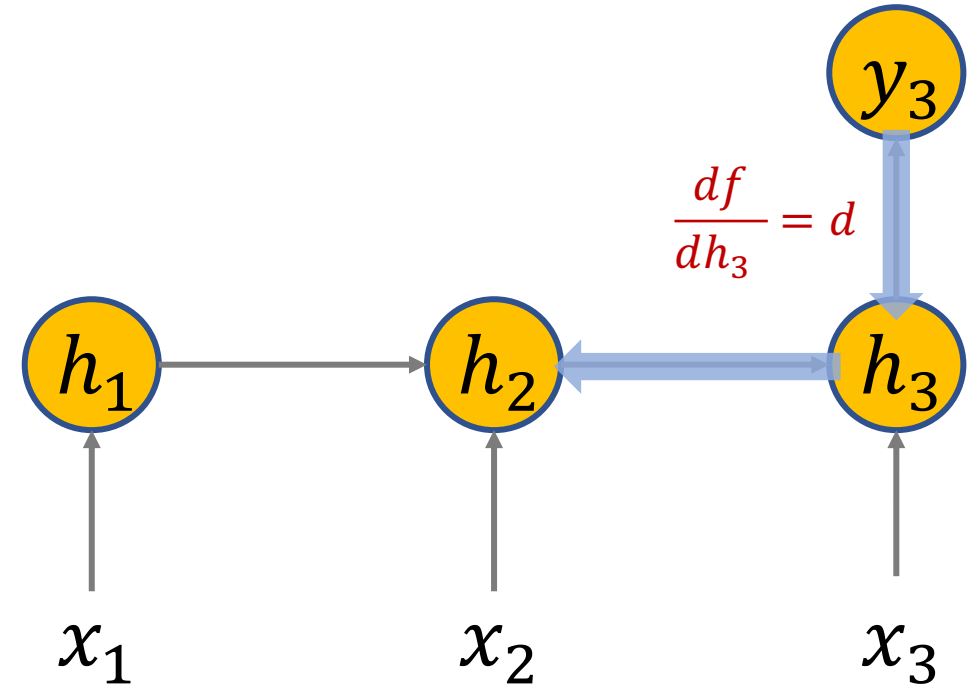
간단한 RNN 학습 예시

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$

Backpropagation

- Let $f = y_3$
- $\frac{df}{dh_1} = \frac{dh_2}{dh_1} \times \frac{dh_3}{dh_2} \times d$
- $\max_x \left(\frac{d}{dx} \tanh(x) \right) = \frac{1}{2}$



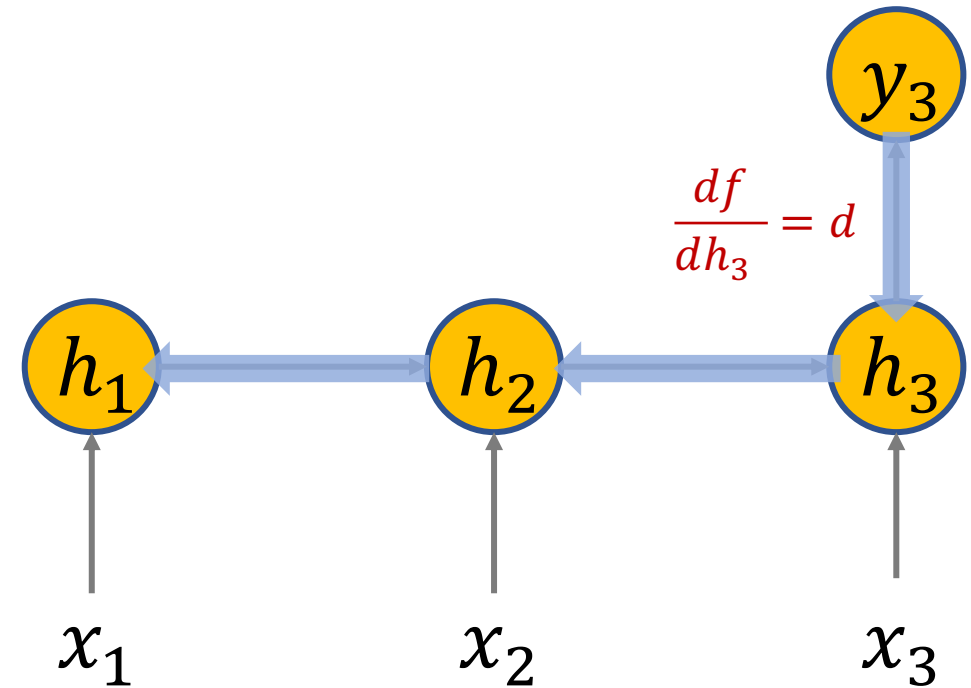
간단한 RNN 학습 예시

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$

Backpropagation

- Let $f = y_3$
- $\frac{df}{dh_1} \leq \frac{dh_2}{dh_1} \times a \times \frac{1}{2} \times d$
- $\max_x \left(\frac{d}{dx} \tanh(x) \right) = \frac{1}{2} \Rightarrow \left| \frac{dh_3}{dh_2} \right| \leq a \times \frac{1}{2}$



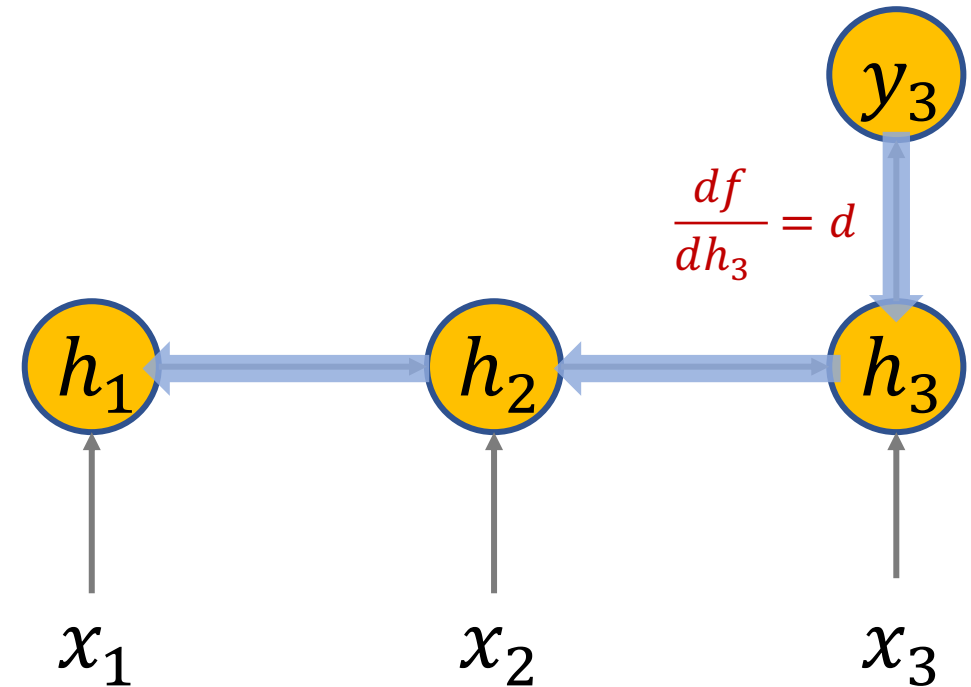
Exploding / Vanishing Gradient

Forward propagation

- $h_1 = \tanh(ah_0 + bx_1 + c)$
- $h_2 = \tanh(ah_1 + bx_2 + c)$
- $h_3 = \tanh(ah_2 + bx_3 + c)$
- $y_3 = dh_3 + e$

Backpropagation

- Let $f = y_3$
- $\frac{df}{dh_1} \leq \boxed{a} \times \frac{1}{2} \times \boxed{a} \times \frac{1}{2} \times d$
- $|a| > 1 \Rightarrow \lim_{n \rightarrow \infty} a^n = \infty \rightarrow \text{Exploding Gradient}$
- $|a| < 1 \Rightarrow \lim_{n \rightarrow \infty} a^n = 0 \rightarrow \text{Vanishing Gradient}$



Exploding / Vanishing Gradient

$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b})$$

- 위 수식은 첫 Hidden-state vector에 W_{hh} 를 반복적으로 곱하는 것과 유사

$$\mathbf{h}_t \propto W_{hh}^{t-1}\mathbf{h}_1$$

- 이 때, W_{hh} 가 **Eigendecomposition**을 통해 VDV^{-1} 로 분해될 수 있다고 가정:

$$W_{hh}^{t-1}\mathbf{h}_1 = VD^{t-1}V^{-1}\mathbf{h}_1$$

- 행렬 D 는 Eigenvalue의 대각 행렬이므로, Eigenvalue의 지수승을 곱함

$$\begin{bmatrix} 2 & 0 \\ 0 & 0.3 \end{bmatrix}^{t-1} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 2^{t-1} & 0 \\ 0 & 0.3^{t-1} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}$$

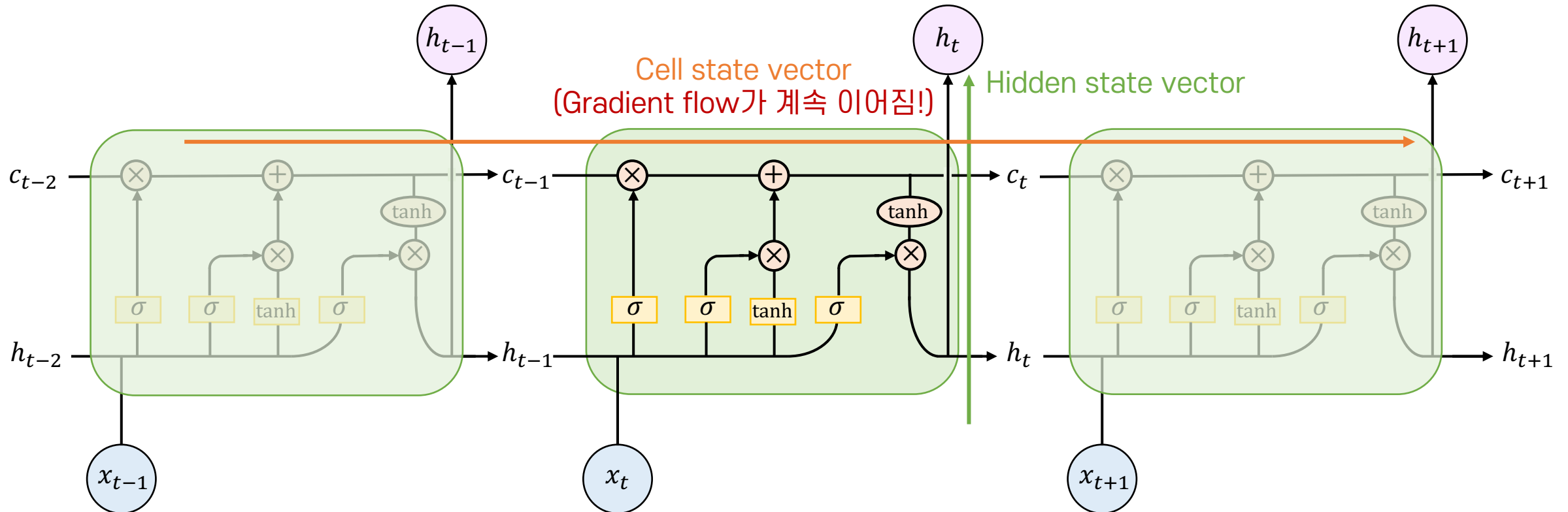
- 따라서 가중치 행렬 W_{hh} 를 **Eigendecomposition** 했을 때,

- Eigenvalue의 절대값이 1 보다 크다 → **Exploding Gradient**
- Eigenvalue의 절대값이 1 보다 작다 → **Vanishing Gradient**

LSTM (Long Short-Term Memory)

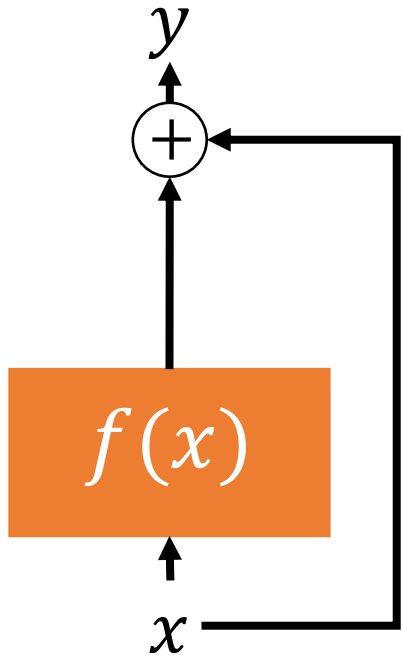
LSTM의 구조

- RNN과 같이 매 Time step마다 반복적으로 동일 모듈을 적용
- LSTM안에는 4개의 Vector를 계산하는 신경망이 존재



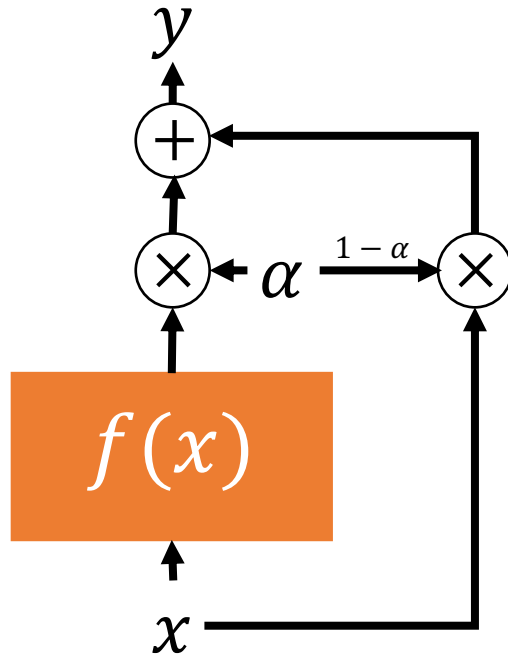
ResNet에서 LSTM까지

$$y = x + f(x)$$



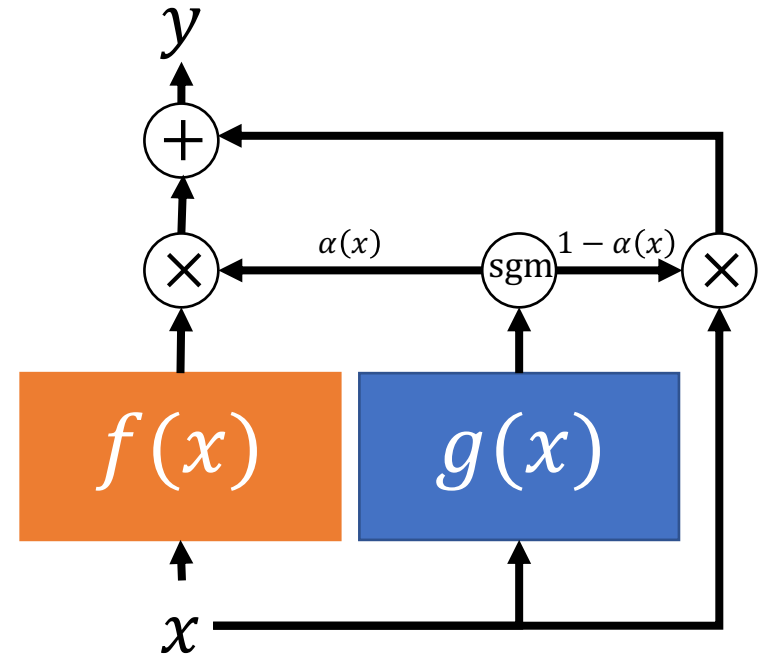
α 가 입력 x 와 출력 $f(x)$ 의
Convex sum을 조절함

$$y = (1 - \alpha)x + \alpha f(x)$$
$$0 < \alpha < 1$$



α 가 동적으로 입력 x 로 결정
이러한 구조를 **Highway network**라 부름

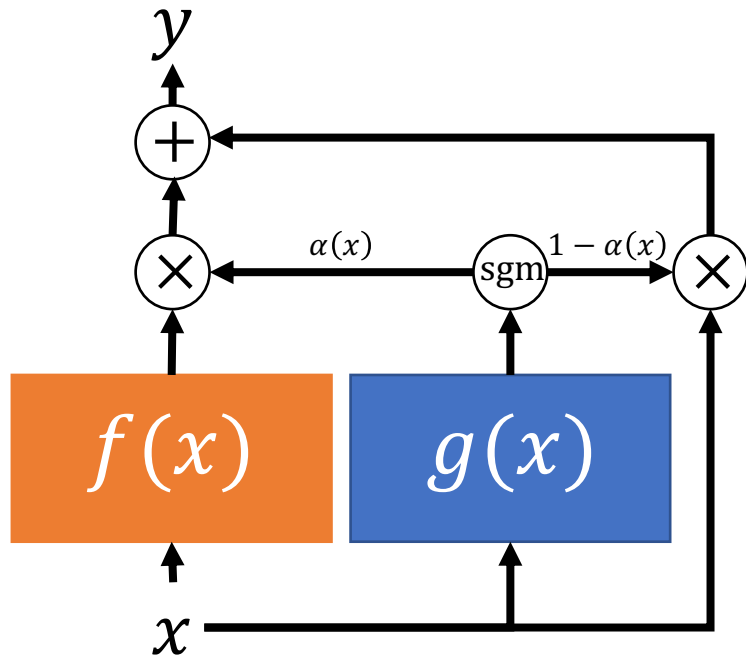
$$y = (1 - \alpha(x))x + \alpha(x)f(x)$$
$$\alpha(x) = \text{sgm}(g(x))$$



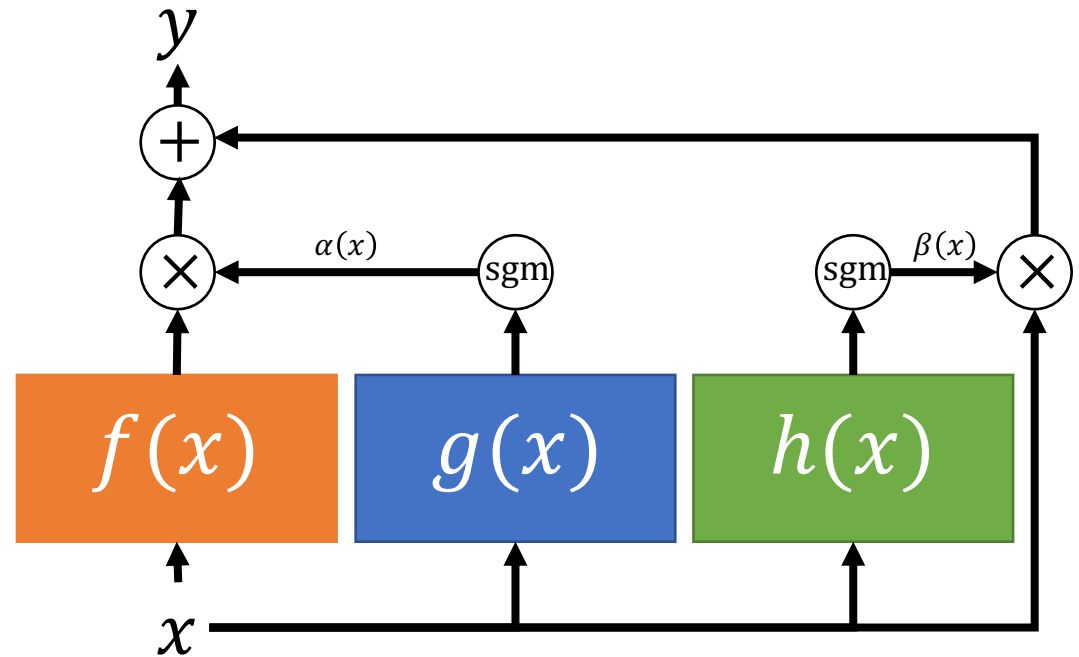
ResNet에서 LSTM까지

$\alpha(x)$ 가 출력 $f(x)$ 을 조절하고 $\beta(x)$ 가 입력 x 를 조절

$$y = (1 - \alpha(x))x + \alpha(x)f(x)$$
$$\alpha(x) = \text{sgm}(g(x))$$



$$y = \beta(x)x + \alpha(x)f(x)$$
$$\alpha(x) = \text{sgm}(g(x)), \beta(x) = \text{sgm}(h(x))$$



ResNet에서 LSTM까지

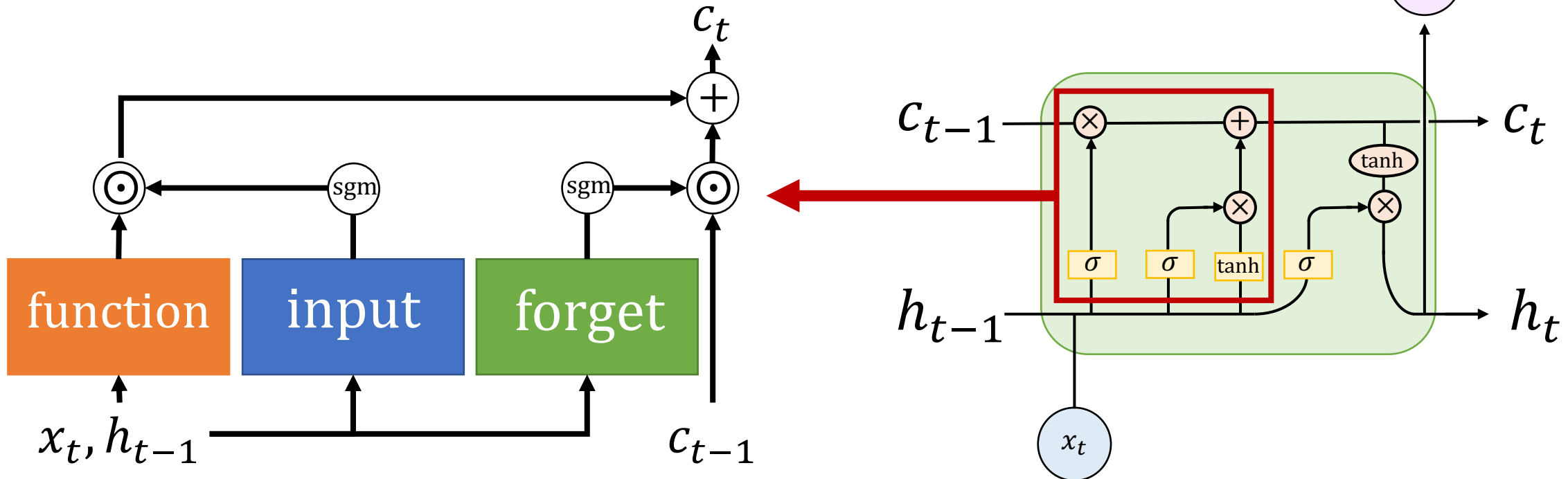
input gate가 현재 Time step에서 계산한 정보의 출력을 조절하고, **forget gate**가 c_{t-1} 을 조절

$$c_t = f(x_t, h_{t-1}) \odot c_{t-1} + i(x_t, h_{t-1}) \odot g(x_t, h_{t-1})$$

$$f(x_t, h_{t-1}) = \text{sgm}(W_f[x_t; h_{t-1}]), \quad i(x_t, h_{t-1}) = \text{sgm}(W_i[x_t; h_{t-1}])$$

$$g(x_t, h_{t-1}) = \tanh(W_g[x_t; h_{t-1}])$$

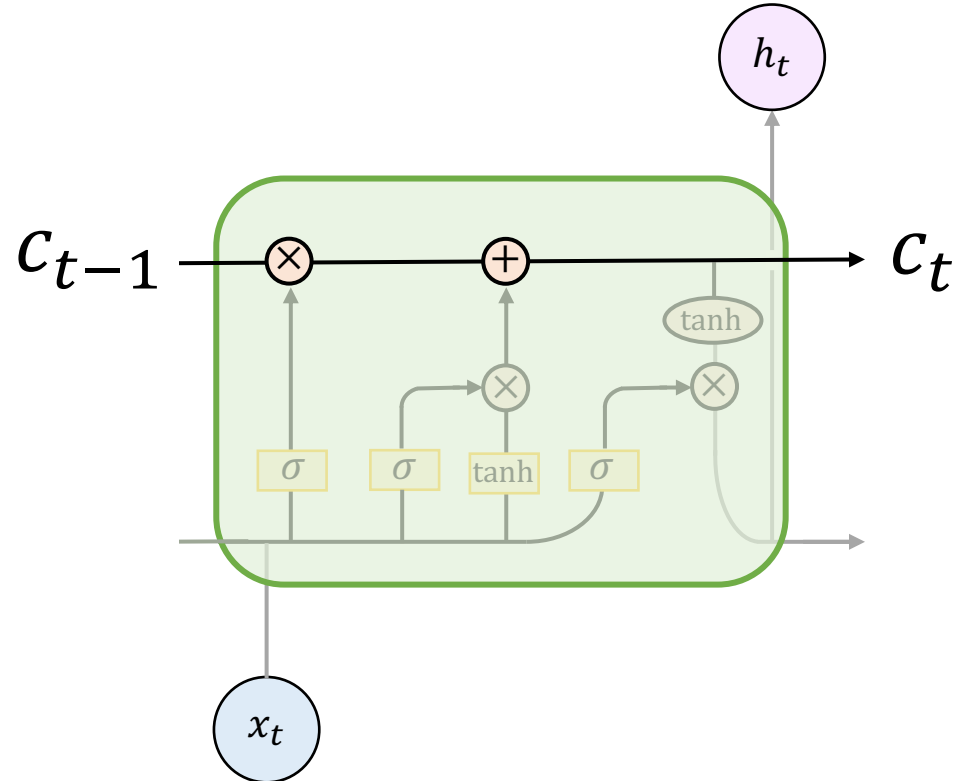
$\odot$: Element-wise Multiplication



LSTM의 구조

핵심 아이디어

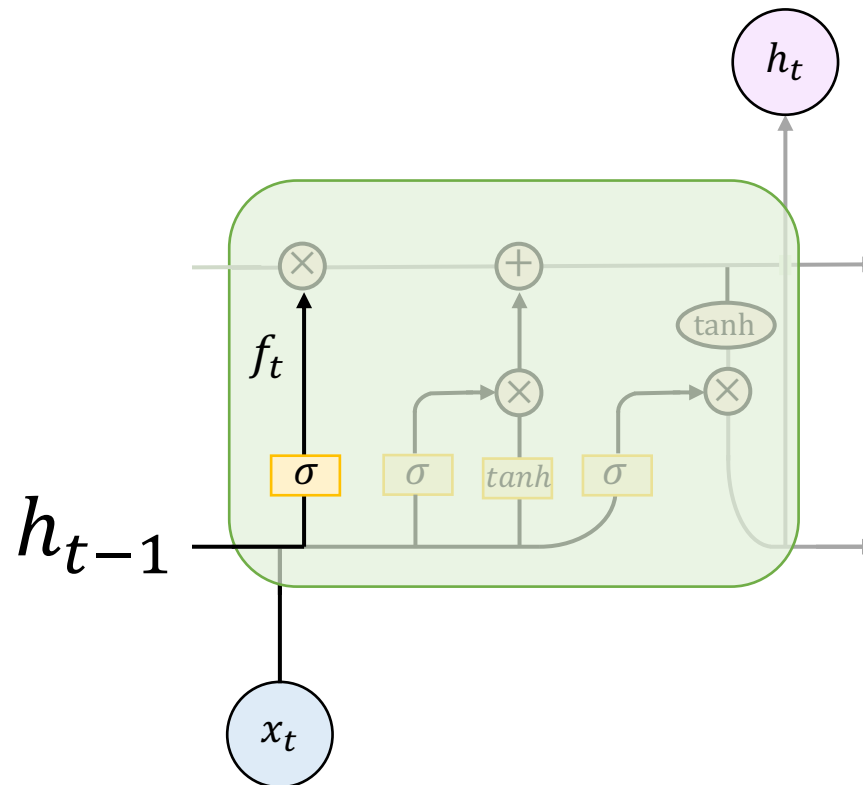
- Cell state 정보를 어떠한 왜곡 없이 전달할 수 있는 능력을 가짐
→ 장기 기억 문제(Long-term dependency)를 해결



LSTM의 구조

- Forget gate를 통해 얼마나 이전 Cell state를 기억할지 결정

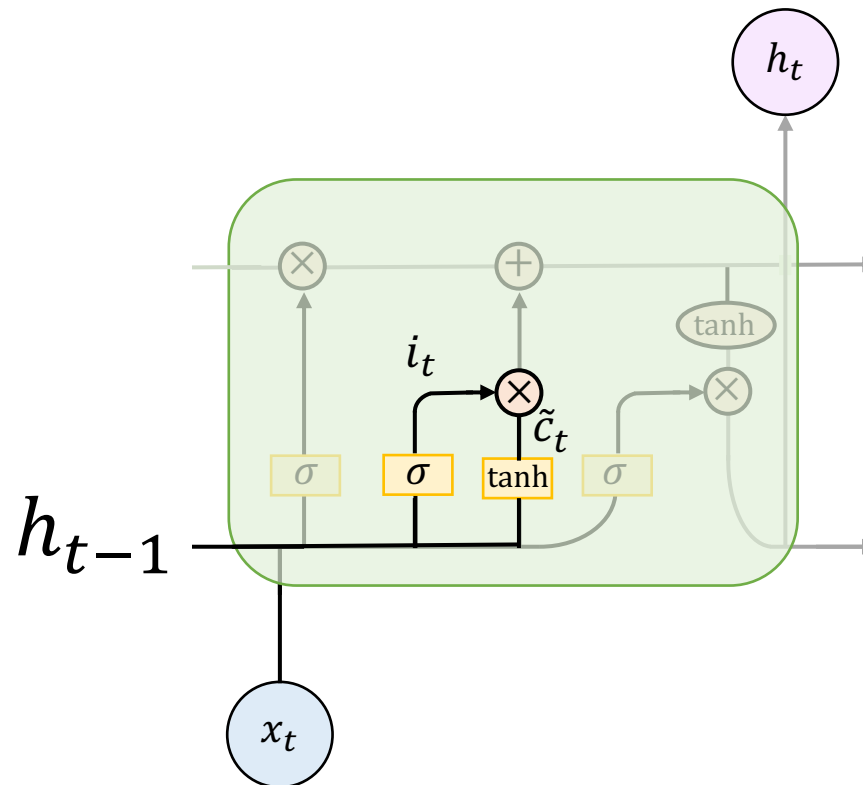
$$f_t = \sigma(W_f[h_{t-1}; x_t] + b_f)$$



LSTM의 구조

- 정보를 생성하고 이를 **Input gate**로 잘라내어 추가

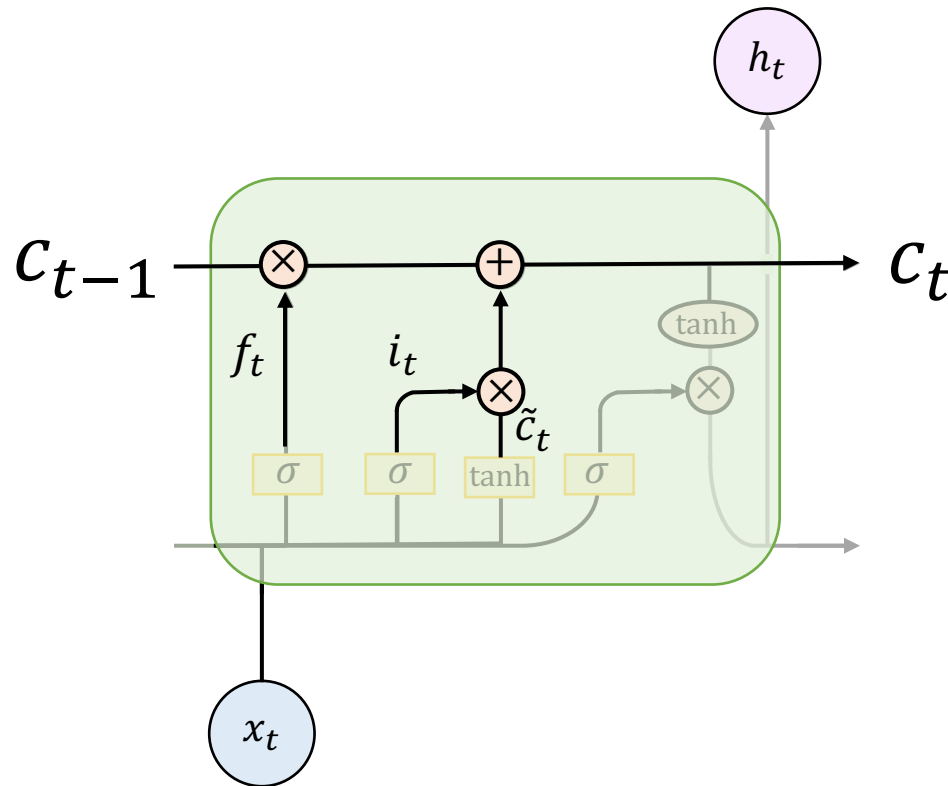
$$i_t = \sigma(W_i[h_{t-1}; x_t] + b_i), \quad \tilde{c}_t = \tanh(W_g[h_{t-1}; x_t] + b_g)$$



LSTM의 구조

- 현재 Cell state를 계산

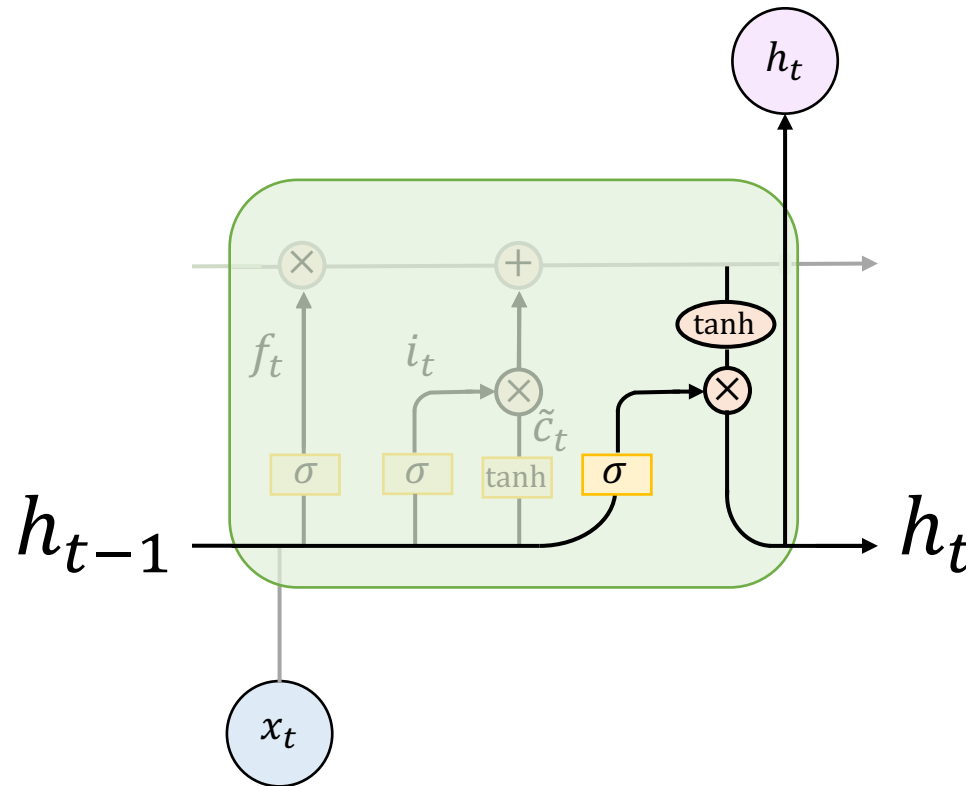
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$



LSTM의 구조

- Cell state로부터 Hidden state를 생성하고, 이는 **Output gate**를 통해 조절
- Hidden state를 다음 Time step으로 전달, 그리고 출력/다음 층으로 전달

$$o_t = \sigma(W_o[h_{t-1}; x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$



Gated Recurrent Unit (GRU)

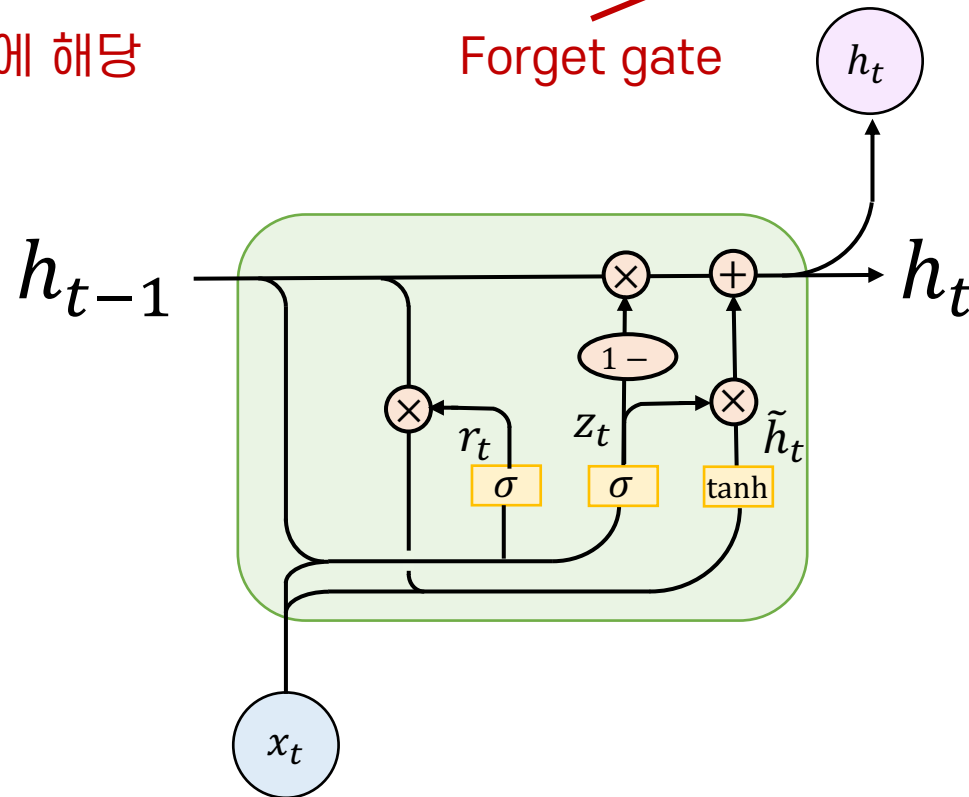
- Cell state와 Hidden state를 하나의 Vector로 통합함

$$z_t = \sigma(W_z[h_{t-1}; x_t]), \quad r_t = \sigma(W_r[h_{t-1}, x_t])$$
$$\tilde{h}_t = \tanh(W[\tilde{r}_t \odot h_{t-1}; x_t]), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

LSTM의 Output gate에 해당

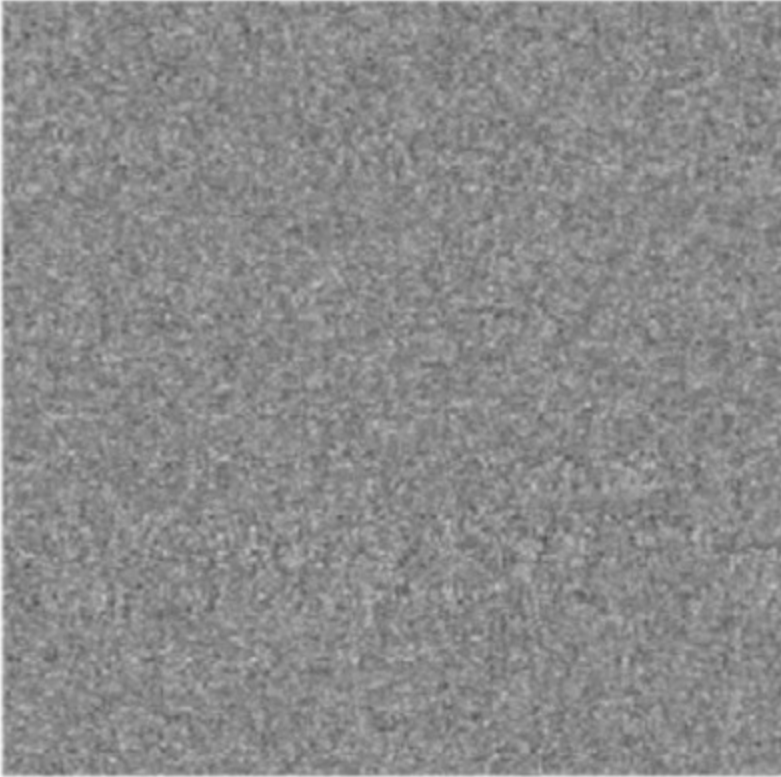
Forget gate

Input gate

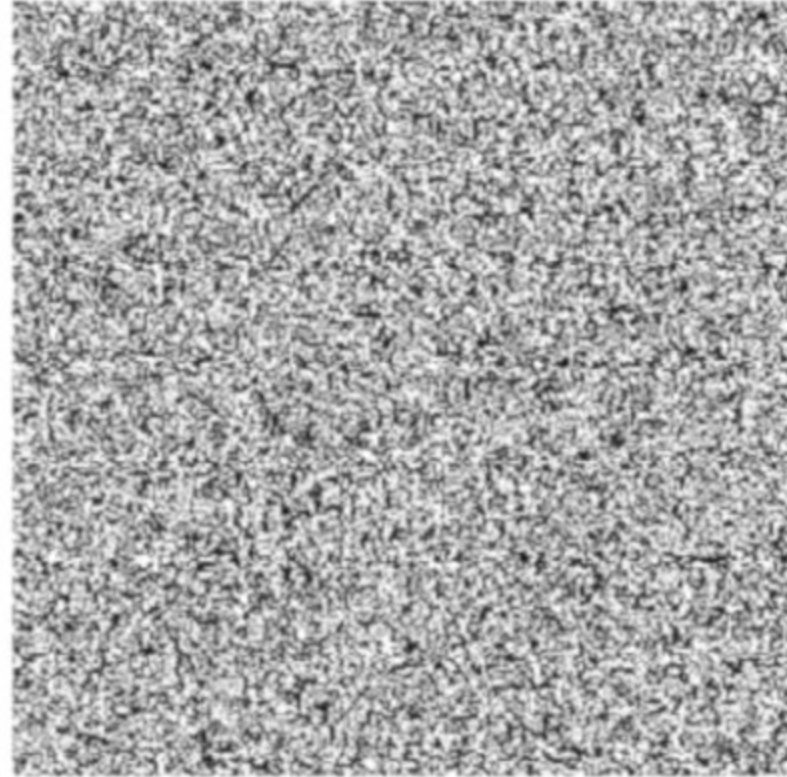


Vanishing Gradient 시각화

127

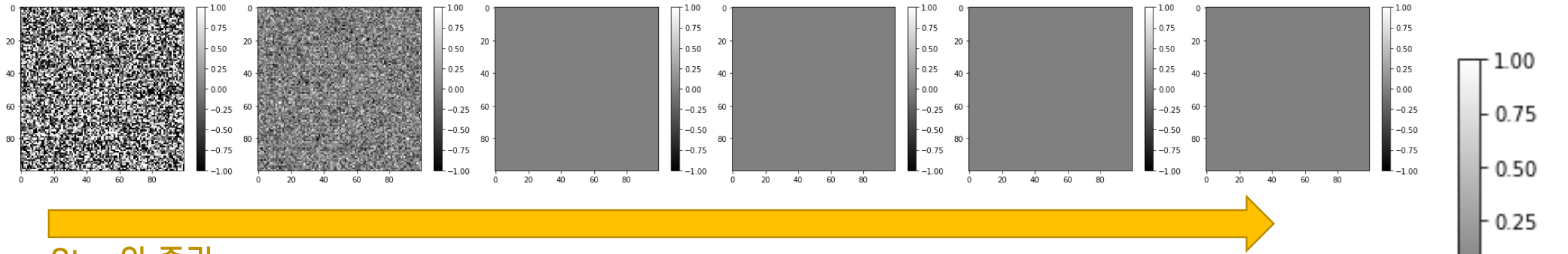


127



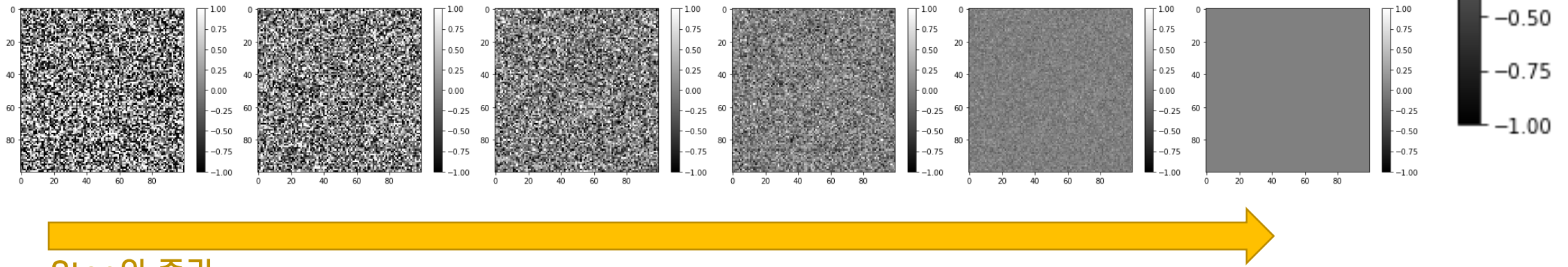
Vanishing Gradient 시각화

RNN의 Gradient 시각화



Step의 증가

해결 방안: LSTM



Step의 증가

Basic NLP Components: RNN, LSTM, and GRU

- Q2) 다음 RNN과 LSTM, GRU 모듈에 관한 설명 중 옳지 않은 것은?
 - RNN은 시점별로 각 시점의 은닉 상태(hidden state)를 다음 시점으로 전달하기 때문에, 이전 시점들의 문맥 정보를 활용할 수 있다.
 - LSTM의 셀 상태(cell state)는 기존 RNN의 은닉 상태와 다르게, 시퀀스가 길어질 때 정보 흐름이 쉽게 소실되지 않도록 설계되었다.
 - RNN을 개선한 LSTM에서는 RNN과 달리 'Vanishing Gradient' 문제가 발생하지 않는다.
 - 문제가 완화되었으나, 완전히 사라진 것은 아님.
 - GRU는 입력 게이트와 망각 게이트를 하나의 업데이트 게이트로 통합하여, LSTM 대비 파라미터 수가 조금 더 적다.

MLE and MAP

- Q3) 최대사후확률추정 (Maximum A Posteriori; MAP)과 최대가능도추정 (Maximum Likelihood Estimation; MLE)의 차이에 대한 설명으로 옳은 것은?
 - MAP는 prior를 반영하지만 MLE는 prior를 반영하지 않는다.
 - MLE와 MAP는 데이터가 많아질수록 다른 결과를 낸다.
 - MLE는 MAP에서 prior를 가우시안 분포로 설정하는 것과 동일하다.
 - 데이터가 많을수록 MAP는 prior의 영향이 크고, MLE는 데이터에만 영향을 받는다.

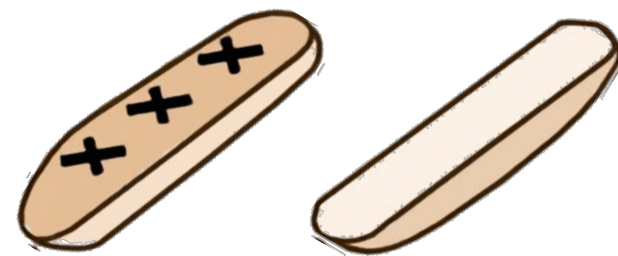
윷이 앞면이 나올 확률은?

- 윷이 앞면이 나올 확률은 거의 0.5이지만 완전히 0.5는 아닐 것
→ 정확한 확률을 알기 위하여 윷을 던져서 앞면이 나올 확률을 알아보자

- 하나의 윷을 10회 던졌을 때 결과가 다음과 같이 나왔다
HHTHTTHHHT (H: Head 앞면, T: Tail 뒷면)

- 위 실험에 기반해서, 윷이 앞면이 나올 확률은?

$$\frac{\text{앞면이 나온 횟수}}{\text{전체 시행 횟수}} = \frac{6}{10} = 0.6$$



- 이때, 윷이 앞면이 나올 확률을 0.6이라 할 수 있을까?

윷 던지기 예제

- 윷의 확률적 모델 (앞면이 나올 확률을 θ 로 가정):

$$P(x|\theta) = \begin{cases} \theta, & x = H \\ 1 - \theta, & x = T \end{cases}$$

$$P(X|\theta) = P(x_1, x_2, \dots, x_n|\theta) = P(x_1|\theta) P(x_2|\theta) \cdots P(x_n|\theta)$$

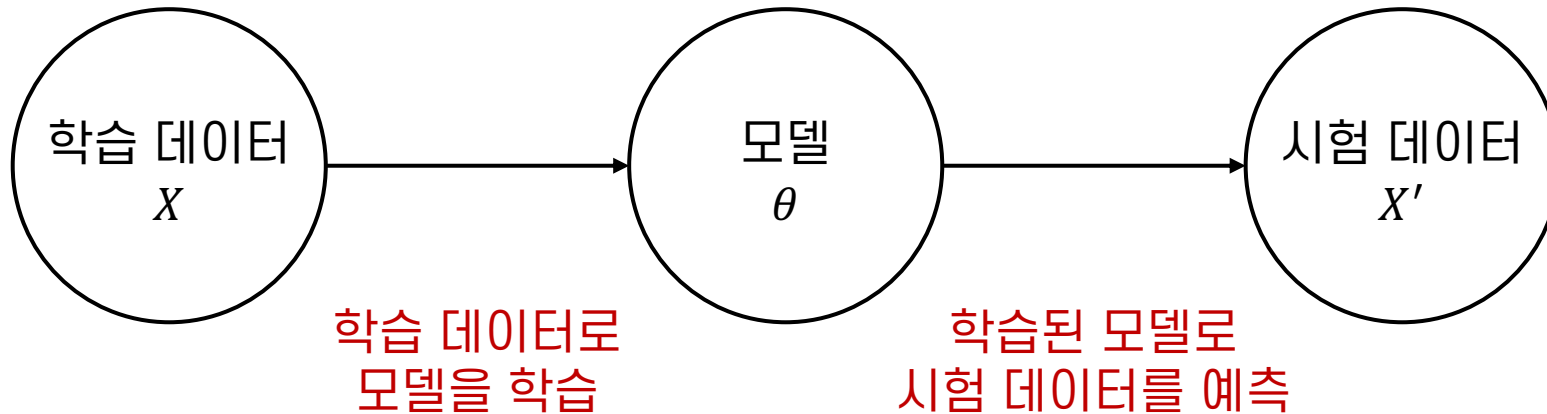
- θ 를 알고 있다면:
 - 윷을 10번 던져 $x = \text{HHTHTTHHHT}$ 가 나올 확률을 계산할 수 있음
 - 윷을 던졌을때 나오는 결과에 해당하는 x 의 분포를 그려볼 수 있음 $\rightarrow P(X|\theta)$

그러나, 우리가 궁금한 것은 $x = \text{HHTHTTHHHT}$ 가 나왔을 때의 θ 값: $P(\theta|X)$

- θ 는 일종의 분포를 나타냄
- 이때, x 가 주어졌을 때 가장 확률 값이 높은 θ 를 찾는 방법은?

베이지안 관점의 기계학습

- 학습 데이터 x 로 모델 θ 를 학습하여 시험 데이터 x' 에서 잘 맞추는 것이 목표
- 베이지안 관점의 기계 학습: x, x' 뿐만 아니라 θ 또한 확률 변수로 생각



베이지안 관점의 기계학습

목표: 학습 데이터 X 가 주어질 때 이를 가장 잘 나타낼 수 있는 $\hat{\theta}$ 를 찾는 것

$$\hat{\theta} = \operatorname{argmax}_{\theta} P(\theta|X)$$

- $X = \text{HHTHTT HHHT}$ 가 나왔을 때 윷을 모델링할 가장 타당한 θ 를 찾자!
- 그러나 $P(\theta|X)$ 를 모델링하는 함수를 찾기는 어려움
→ 대신, Bayes' Rule을 이용하여 $P(X|\theta)$ 를 활용하자!

$$\begin{aligned}\hat{\theta} &= \operatorname{argmax}_{\theta} P(\theta|X) = \operatorname{argmax}_{\theta} \frac{P(X|\theta)P(\theta)}{P(X)} \\ &= \operatorname{argmax}_{\theta} \underbrace{P(X|\theta)}_{\text{Likelihood}} \underbrace{P(\theta)}_{\text{Prior}}\end{aligned}$$

$P(X)$ 는 θ 와 관련이 없음

Maximum Likelihood Estimation

Likelihood의 정의:

$$\text{Likelihood} := P(X|\theta)$$

- 해석1: θ 를 가정했을 때 학습 데이터 x 가 얼마나 잘 모델링 되었는가?
- 해석2: 베이지안 관점의 기계학습 모델링에서

$$\hat{\theta} = \operatorname{argmax}_{\theta} P(X|\theta)P(\theta)$$

$P(\theta)$ 를 상수로 가정 /
Prior를 Uniform distribution으로 가정

⇒ 어떤 θ 도 가질 수 있다고 생각하는 관점

- θ 가 0.1일수도 0.7일수도 있으며, 딱히 우리의 편견이 들어가지 않음
- Likelihood가 높다 ⇒ 학습 데이터를 잘 모델링 했다
- Maximum Likelihood Estimation (MLE): Likelihood가 최대인 θ 를 찾기

$$\hat{\theta}_{MLE} = \operatorname{argmax}_{\theta} P(X|\theta)$$

Maximum A Posteriori Estimation: 윷 던지기 예제

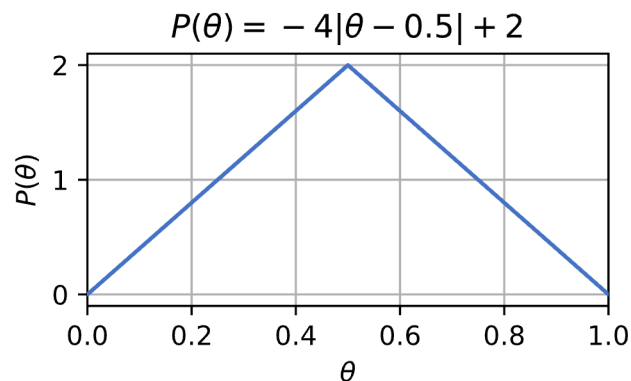
- 하나의 윷을 10회 던졌을 때 결과가 다음과 같이 나왔다
 - **HH****T****HT****TT****HH****HT** (H: Head 앞면, T: Tail 뒷면)

윷의 수학적 모델:

$$P(x|\theta) = \begin{cases} \theta, & x = H \\ 1 - \theta, & x = T \end{cases}$$

$$P(X|\theta) = P(x_1, x_2, \dots, x_n|\theta) = P(x_1|\theta) \times P(x_2|\theta) \times \dots \times P(x_n|\theta)$$

- 사전 분포 $P(\theta) = -4|\theta - 0.5| + 2$



앞면이 나올 확률이 0.5 주변에 있을 가능성이 크니까
0.5일 확률이 가장 높은 임의의 분포로 가정해보자
(다른 원하는 사전 분포를 가정해도 무방함)

Maximum A Posteriori Estimation: 옷 던지기 예제

MAP 관점에서 최적의 θ 는?

$$\begin{aligned}\hat{\theta}_{MAP} &= \operatorname{argmax}_{\theta} P(X|\theta)P(\theta) \\ &= \operatorname{argmax}_{\theta} \theta^6(1-\theta)^4(-4|\theta-0.5|+2)\end{aligned}$$

- $\operatorname{argmax}_{\theta} \theta^6(1-\theta)^4(-4|\theta-0.5|+2)$ 를 찾기 위해 미분

$$\frac{d}{d\theta} \theta^6(1-\theta)^4(-4\theta) = 4\theta^6(11\theta-7)(1-\theta)^3 = 0 \quad \theta \in [0, 0.5]$$

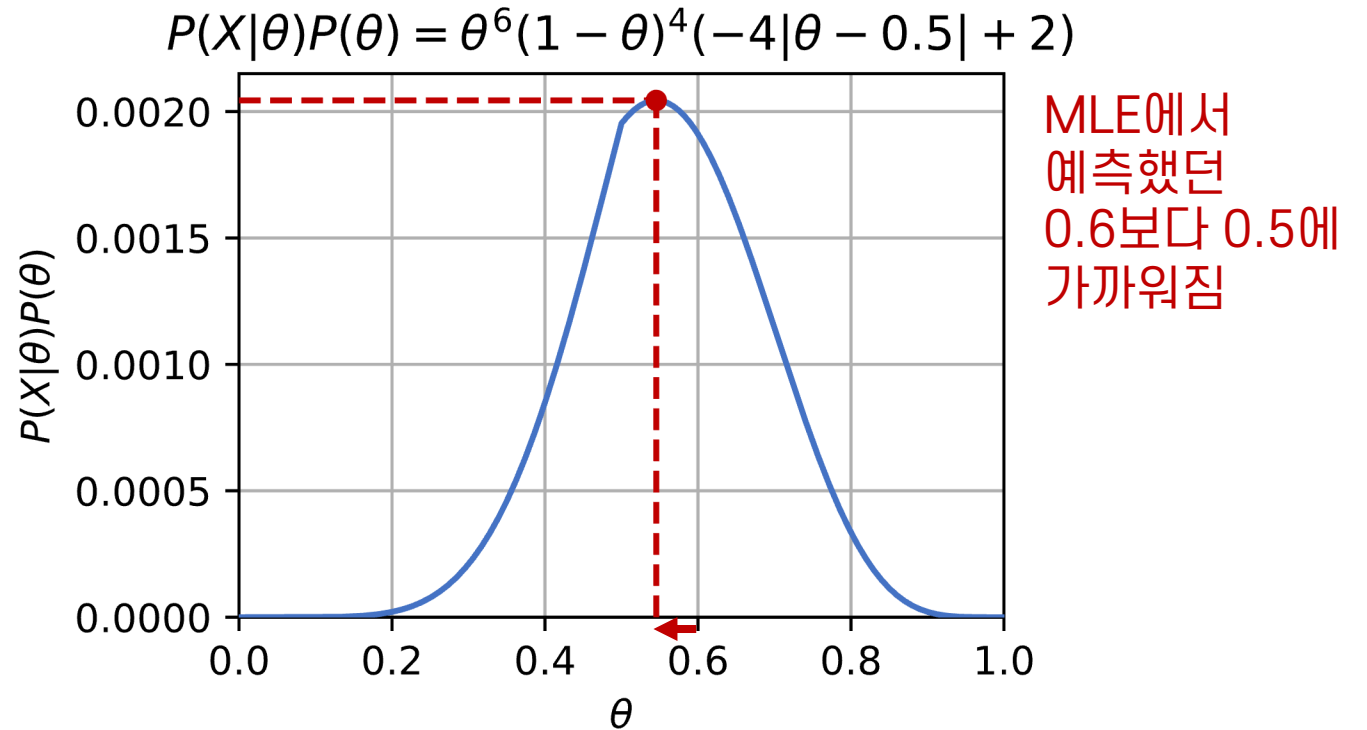
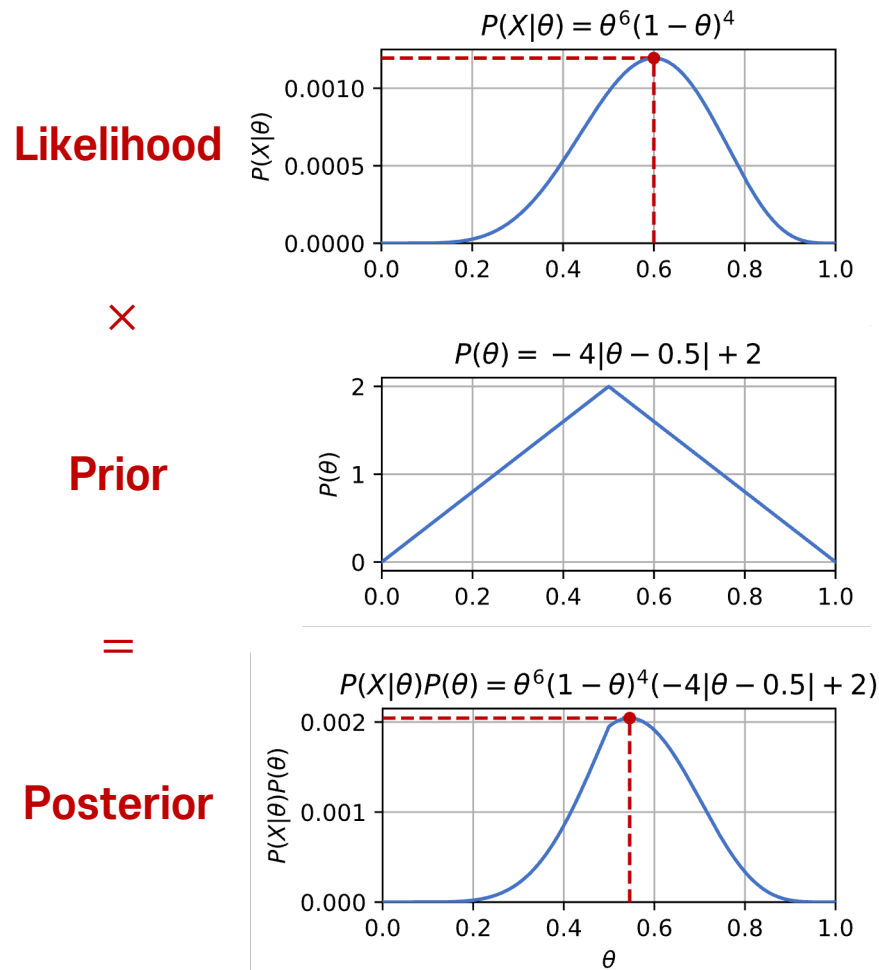
$$\frac{d}{d\theta} \theta^6(1-\theta)^4(-4\theta+4) = -4\theta^5(11-6\theta)(1-\theta)^4 = 0 \quad \theta \in [0.5, 1]$$

대입하면, $\theta = 0, 1$ 인 지점은 극소값, $\theta = \frac{7}{11}$ 은 범위 밖, $\theta = \frac{6}{11}$ 인 지점이 극대

$$\therefore \hat{\theta}_{MAP} = \frac{6}{11}$$

Maximum A Posteriori Estimation: 옷 던지기 예제

- Posterior



MLE vs MAP

MAP의 장점: 적절한 사전분포의 모델링은 과적합으로부터 강건해질 수 있음

- 시행 횟수 / 학습 데이터 개수가 적은 경우, 적절한 Prior를 통해 보정이 가능
- 만약 윷을 3번만 던져서 모델링하는데, 모두 H가 나온 경우,

$$\hat{\theta}_{MLE} = 1, \quad \hat{\theta}_{MAP} = 0.75 \text{ (앞의 예제와 같은 Prior 사용)}$$

MAP의 단점: 과한 (잘못된) 사전 분포의 가정은 모델링에 방해가 될 수 있음

- 윷은 볼록하게 생겨서 앞면이 나올 확률은 실제로 0.5보다 작은 곳에 있을 것

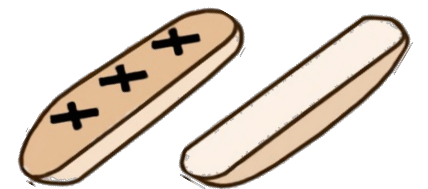
상황에 맞게 적절한 사전 분포의 가정이 필요!

Prior를 고려하지 않는 MLE는 단순히 $\hat{\theta}_{MLE} = 0.6$ 으로, H가 나왔던 확률과 같게 모델링

$$\hat{\theta}_{MLE} = 0.6$$

윷의 특성에 맞추어 Prior를 0.5에 더 높여 준 MAP는 MLE의 예측값과 0.5의 중간에서 모델링

$$\hat{\theta}_{MAP} = \frac{6}{11} = 0.5454 \dots$$



MLE and MAP

- Q3) 최대사후확률추정 (Maximum A Posteriori; MAP)과 최대가능도추정 (Maximum Likelihood Estimation; MLE)의 차이에 대한 설명으로 옳은 것은?
 - MAP는 prior를 반영하지만 MLE는 prior를 반영하지 않는다.
 - MLE와 MAP는 데이터가 많아질수록 다른 결과를 낸다.
 - 데이터가 많아질수록 같은 결과로 수렴한다.
 - MLE는 MAP에서 prior를 가우시안 분포로 설정하는 것과 동일하다.
 - MLE는 MAP에서 prior를 uniform 분포로 설정하는 것과 동일하다.
 - 데이터가 많을수록 MAP는 prior의 영향이 크고, MLE는 데이터에만 영향을 받는다.
 - 데이터가 많을수록 MAP는 likelihood에 집중한다.

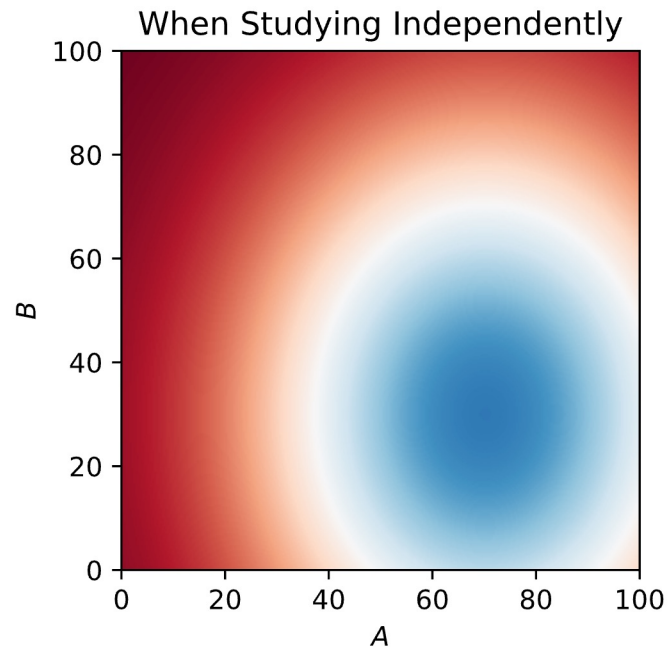
Conditional Distribution

- Q4) 다음 보기 중 conditional과 marginal distribution에 대한 설명으로 옳은 것을 고르시오
 - X 라는 랜덤 변수와 Y 라는 랜덤 변수의 correlation이 0 이라면, $P(X, Y) = P(X)P(Y)$ 이다.
 - $P(X|Y) = P(Y|X)P(Y)/P(X)$
 - $P(X|Y) = P(X, Y)/P(Y)$
 - $P(X) = \sum_y P(X|Y = y)$

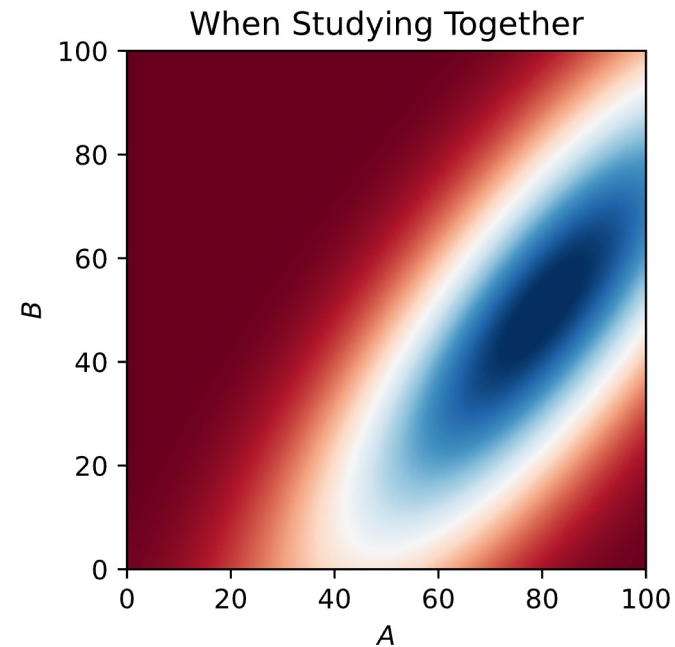
Joint Distribution

Joint distribution: 변수가 2개 이상 존재할 때의 확률 분포 $p(x, y)$

- e.g., 같이 공부한 A, B가 시험을 쳤을 때 둘의 시험 점수에 대한 확률 분포



서로 독립적으로 분포



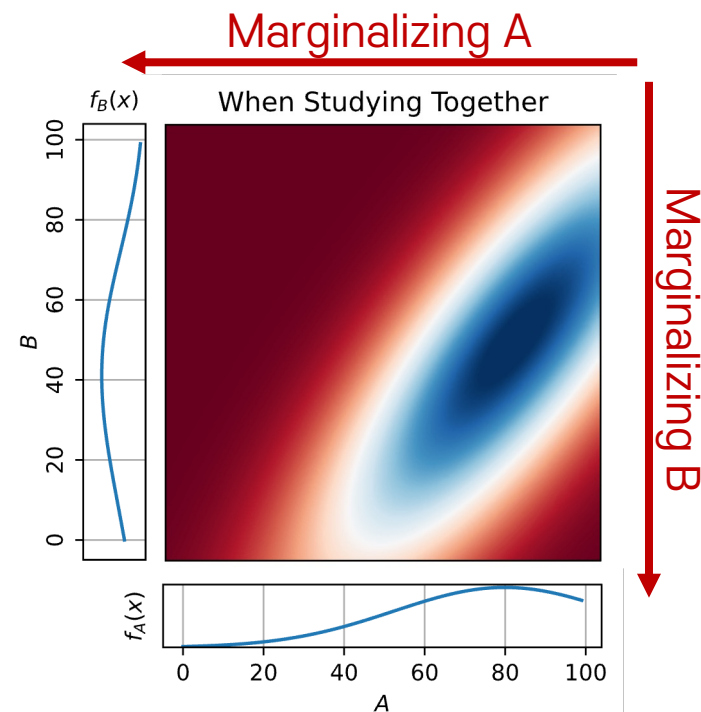
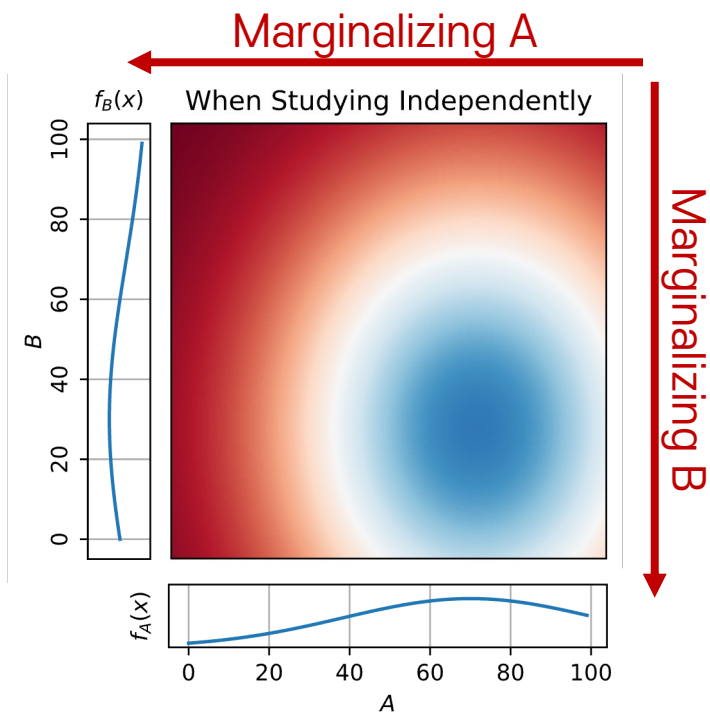
각각 성적이 올랐고
서로의 성적이 오르면 같이 오름

Marginal Distribution

Marginal distribution

- Joint distribution에서 특정 변수만을 고려한 확률 분포
 - Joint distribution에서 Marginal distribution으로 보내는 것을 Marginalization라고 함

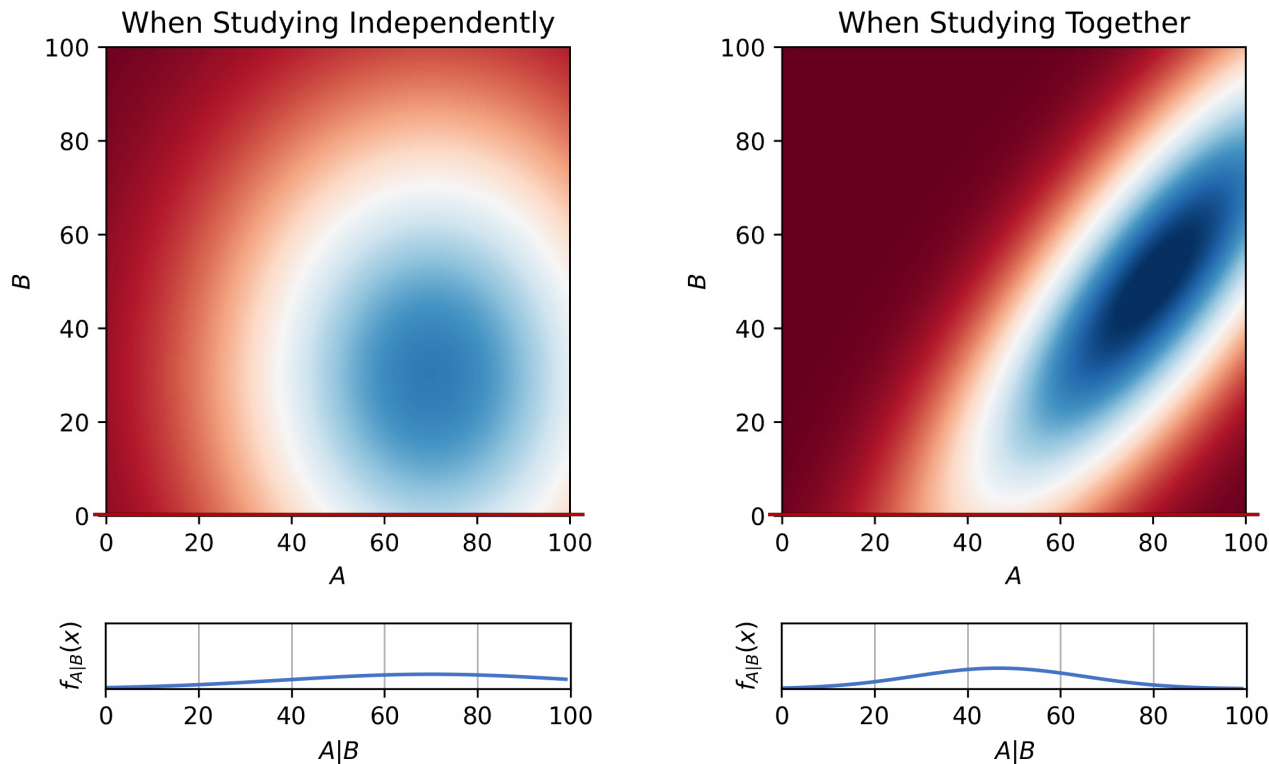
$$p(x) = \int_{-\infty}^{\infty} p(x, y) dy$$



Conditional Distribution

Conditional distribution $P(X|Y)$

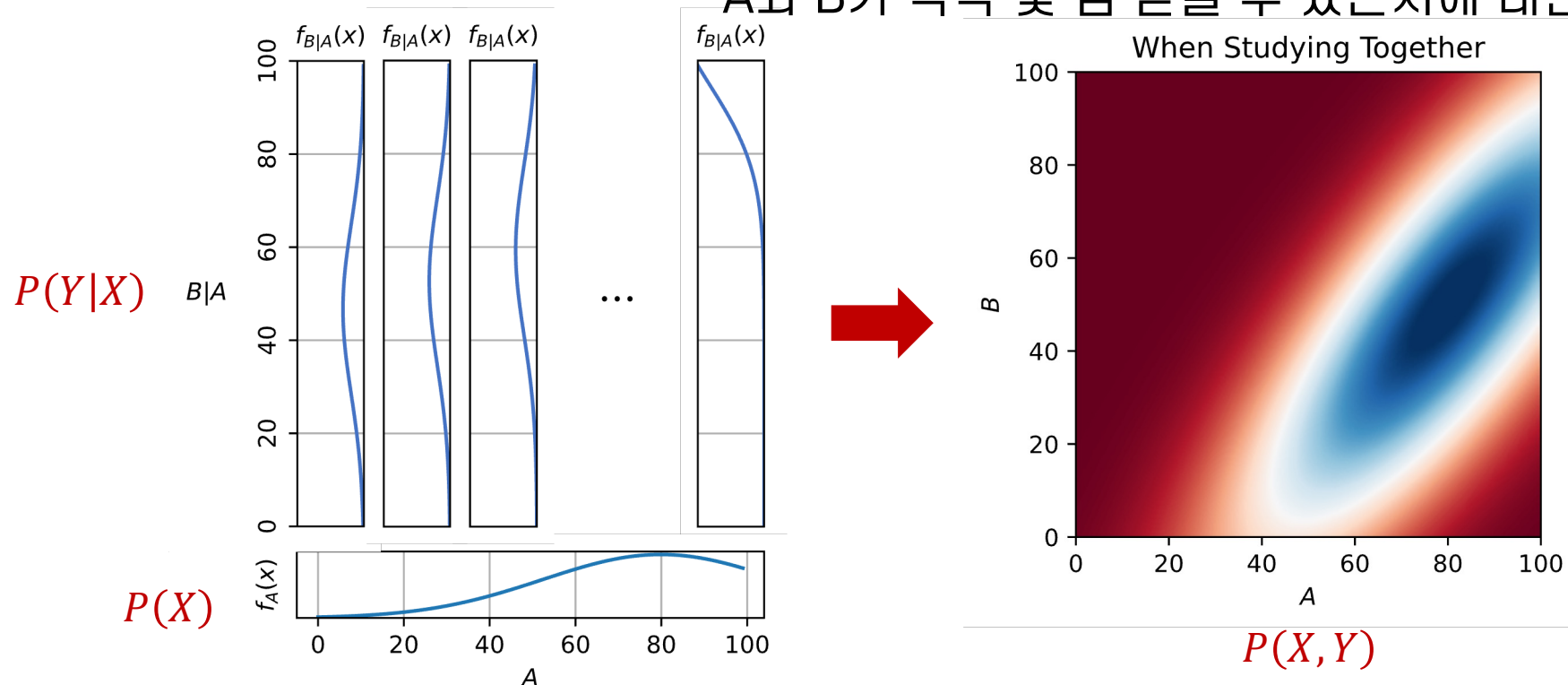
- Joint distribution에서 한 변수의 값을 주었을 때 나머지 변수의 확률 분포
 - E.g., 같이 공부한 Joint distribution에서 B가 100점을 받았을 때,
A의 확률 분포를 보면 100점을 받을 확률이 굉장히 높음



Conditional Distribution

Conditional distribution $P(X|Y)$

- Condition으로 준 Y 의 분포 $P(Y)$ 를 알면 Joint distribution을 구할 수 있음
- $P(X, Y) = P(X|Y) \cdot P(Y)$
 - e.g., B의 점수에 따른 A의 점수 분포를 알고, B의 점수에 대한 분포를 안다면, A와 B가 각각 몇 점 받을 수 있는지에 대한 분포를 알 수 있음



Bayes' Rule

- Conditional distribution의 정의를 이용

$$P(X|Y) \cdot P(Y) = P(X, Y) = P(Y|X) \cdot P(X)$$

Bayes' Rule

$$P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

- $P(Y|X)$ 가 직접적으로 계산할 수 없다면?
→ $P(X|Y)$ 와 $P(Y), P(X)$ 를 이용하여 $P(Y|X)$ 를 간접적으로 계산

Conditional Distribution

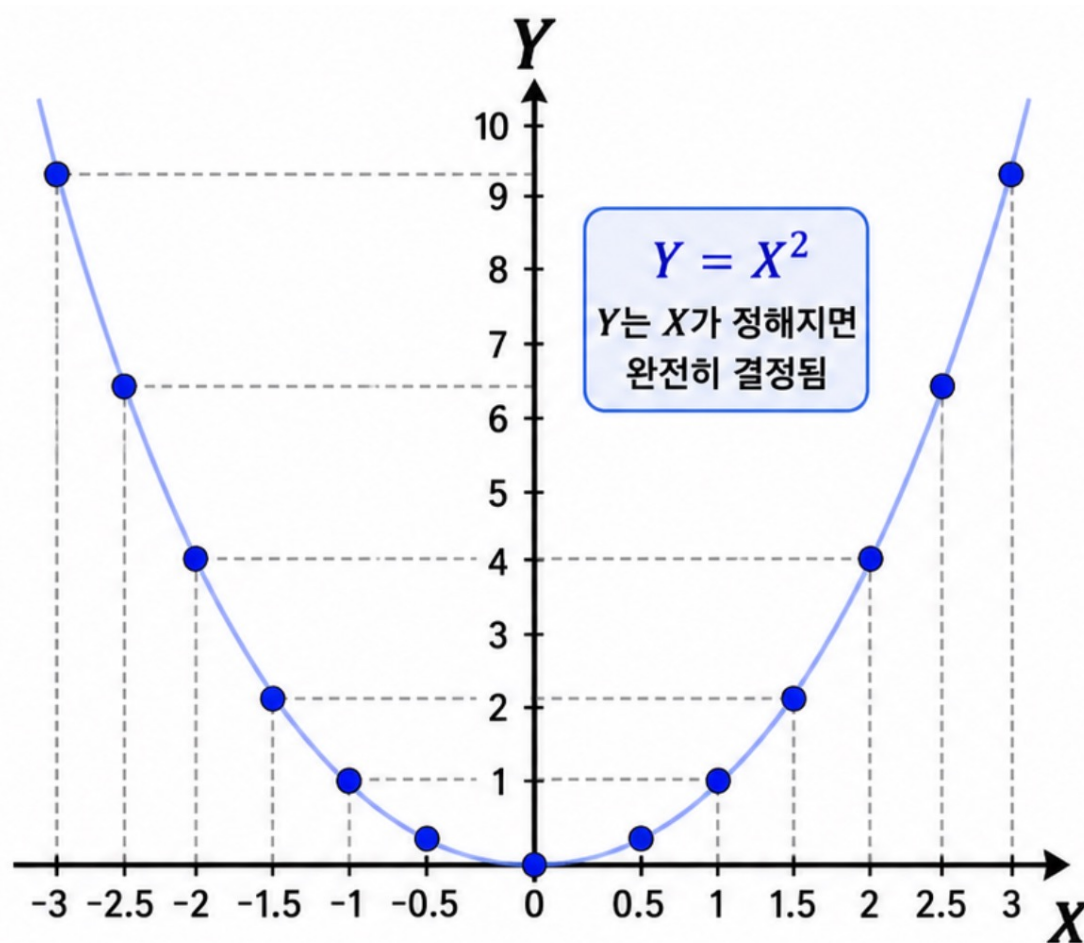
- Q4) 다음 보기 중 conditional과 marginal distribution에 대한 설명으로 옳은 것을 고르시오
 - $P(X|Y) = P(Y|X)P(Y)/P(X)$
 - $P(X|Y) = P(X|Y)P(Y)/P(X)$.
 - $P(X|Y) = P(X, Y)/P(Y)$
 - $P(X) = \sum_y P(X|Y = y)$
 - $P(X) = \sum_y P(X, Y = y)$

Conditional Distribution

- Q4) 다음 보기 중 conditional과 marginal distribution에 대한 설명으로 옳은 것을 고르시오
 - a. X 라는 랜덤 변수와 Y 라는 랜덤 변수의 correlation이 0 이라면, $P(X, Y) = P(X)P(Y)$ 이다.
- 참고) 피어슨 상관계수 (Pearson Correlation Coefficient).
 - $$\rho_{X,Y} = \frac{\mathbb{E}[(X-\mu_X)(Y-\mu_Y)]}{\sqrt{\mathbb{E}[(X-\mu_X)^2]}\sqrt{\mathbb{E}[(Y-\mu_Y)^2]}}$$
 - $\mu_X = \mathbb{E}[X] \quad \mu_Y = \mathbb{E}[Y]$
 - Correlation: 두 변수가 서로 양의 상관 관계를 가지는지 음의 상관관계를 가지는지 나타내는 단일 숫자.

Conditional Distribution

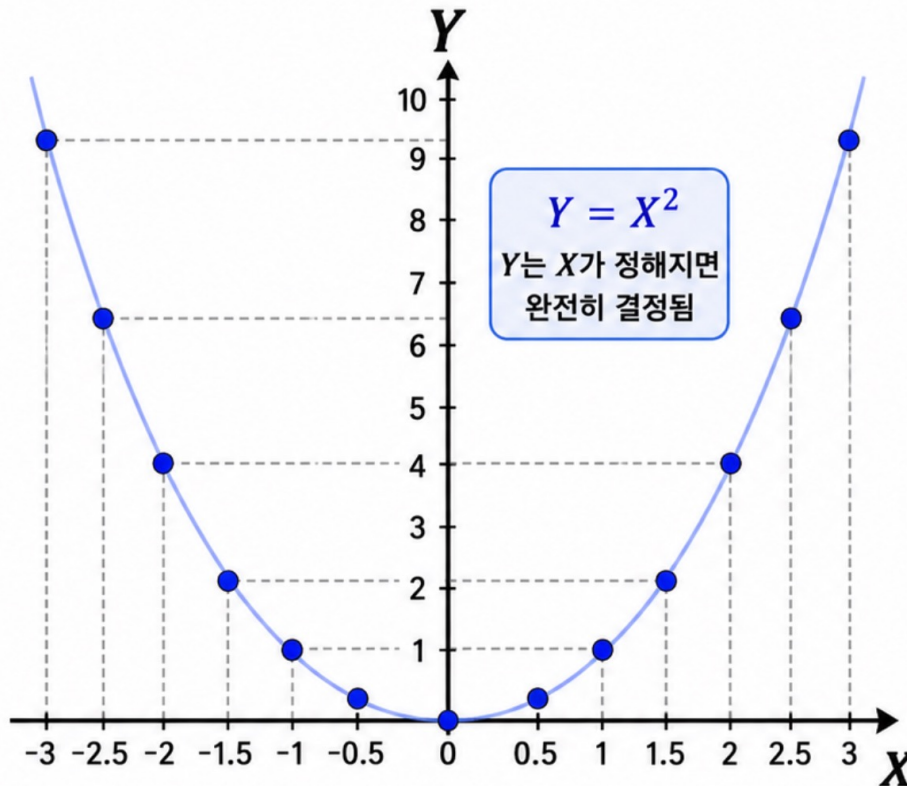
- 대표적 반례: $Y = X^2$
 - Y 는 X 가 정해지면 완전히 결정됨.



그러나 x 의 분포가 0에 대해 대칭.
⇒ X 가 증가할 때 Y 가 증가하기도 하고 감소하기도 함.
⇒ Correlation = 0

Conditional Distribution

- Q4) 다음 보기 중 conditional과 marginal distribution에 대한 설명으로 옳은 것을 고르시오
 - a. X 라는 랜덤 변수와 Y 라는 랜덤 변수의 correlation이 0 이라면, $P(X, Y) = P(X)P(Y)$ 이다.
 - Correlation이 0이라도 independent하지 않을 수 있으므로 오답.

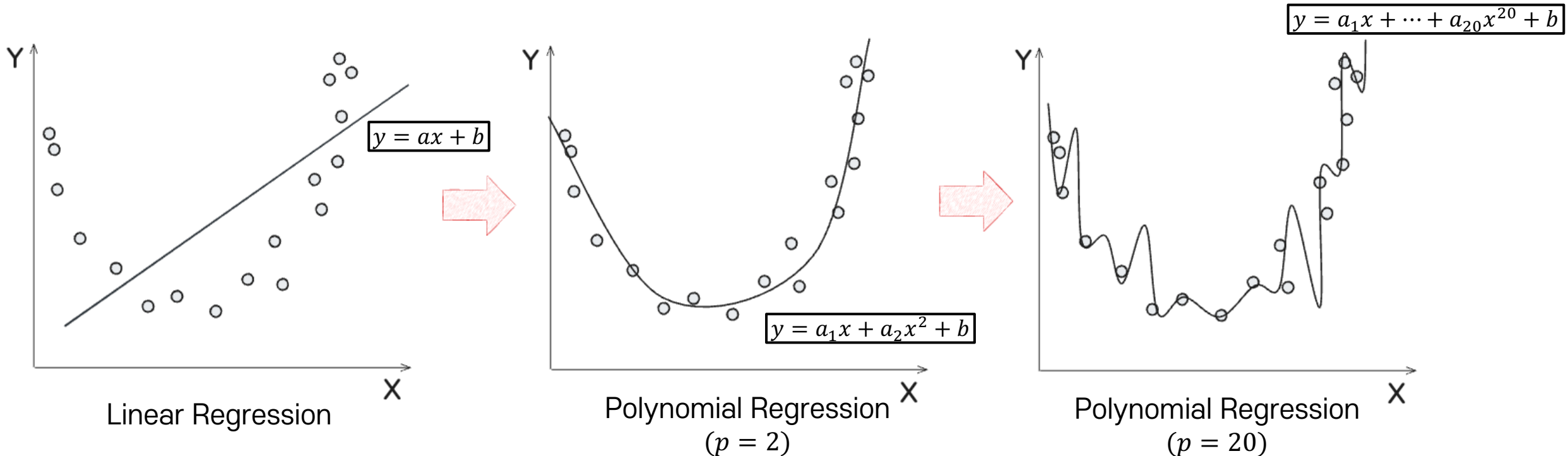


L1, L2 Regularization

- Q5) 머신러닝에서 L1 정규화가 L2 정규화보다 선호되는 경우는 언제인가?
 - L1 정규화는 모델이 결과에 강한 영향을 미치는 희소한(sparse) feature set을 학습할 것으로 예상될 때 선호된다.
 - L1 정규화는 input feature 간 다중공선성(multicollinearity)을 제어해야 할 때 선호된다.
 - L1 정규화는 모델이 overfitting될 가능성이 있고, feature 수를 보존하면서 variance를 줄여야 할 때 선호된다.
 - L1 정규화는 feature 수가 많고 모든 feature가 모델에 고르게 기여하도록 해야 할 때 선호된다.

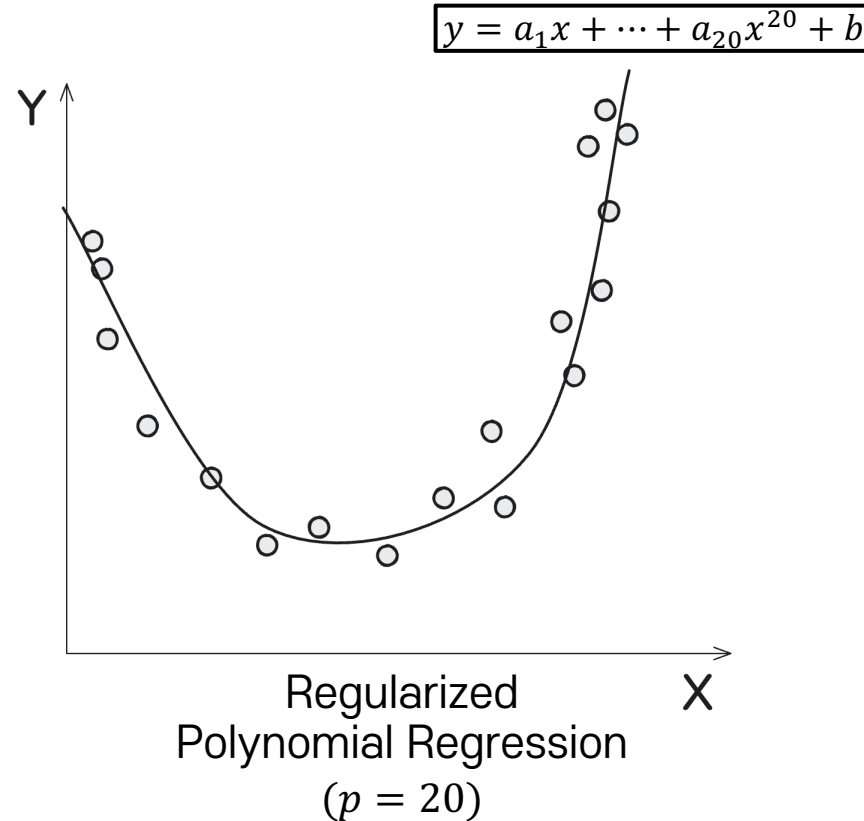
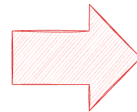
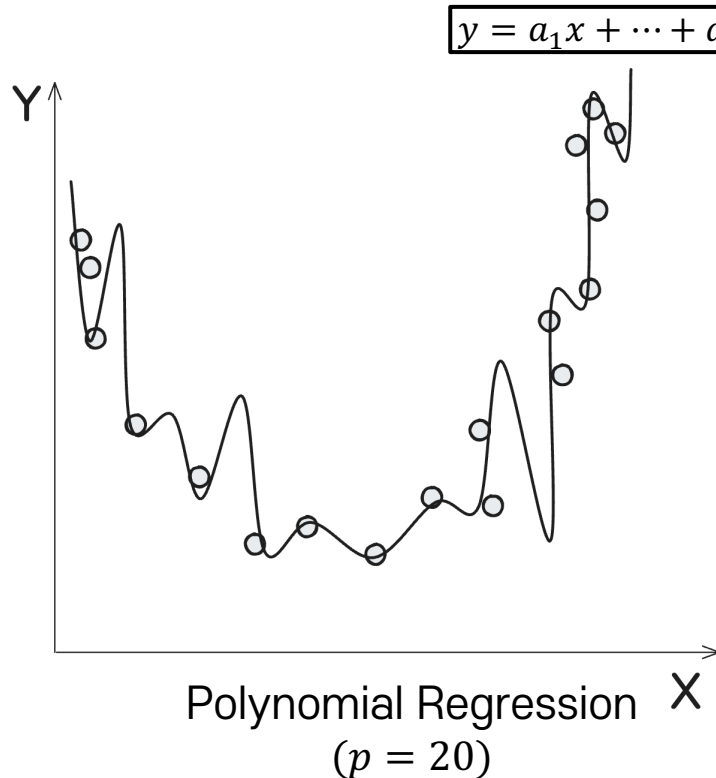
Recap: Overfitting

- Overfitting: 모델이 학습 데이터를 지나치게 학습하여, 새로운 데이터(테스트 셋)에 대한 예측 성능이 떨어지는 현상
 - 학습해야 될 패턴 대비 더 복잡한 모델을 사용 (예: $p = 20$ 인 Regression 모델)
 - 해결책: 덜 복잡한 모델을 사용



Regularization

- 모델 정규화 (Model Regularization)
 - 학습 과정에서 머신러닝 모델의 Overfitting을 줄이기 위한 방법
 - 모델을 학습할 때 특정한 제약 조건을 걸어서 덜 복잡한 모델이 학습되게 함

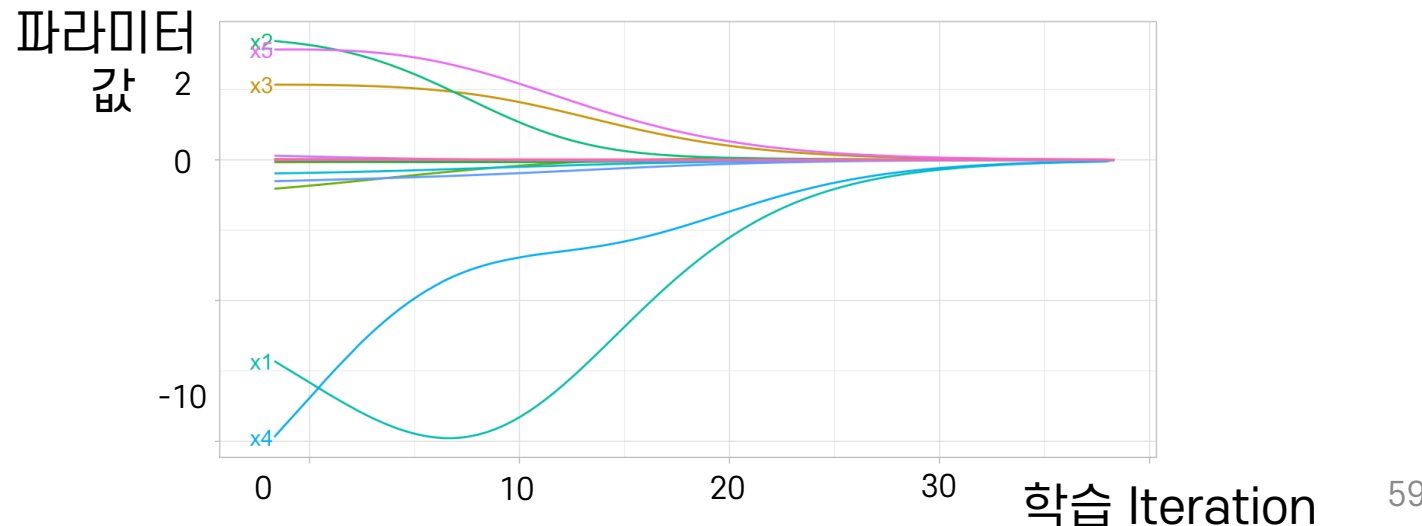


L2 Regularization

- 모델 학습 시 모델 파라미터의 제곱 값이 너무 커지지 않도록 제약
- 예시: Ridge Regression
 - 기존 손실 함수 (MSE): $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (y_i - (a_0 + a_1x + a_2x^2 + \dots + a_{10}x^{10}))^2$
 - 의미: MSE를 작게 하는 $a_1, \dots, a_{10}, b$ 를 학습해라
 - 정규화가 추가된 손실 함수: $\frac{1}{N} \sum_{i=1}^N (y_i - (a_0 + a_1x + a_2x^2 + \dots + a_{10}x^{10}))^2 + \lambda \sum_{j=0}^{10} a_j^2$
 - 의미: MSE를 작게 하면서 파라미터의 제곱 값이 너무 커지지 않도록 하는 $a_0, a_1, \dots, a_{10}$ 를 학습해라
 - $\sum_{j=0}^{10} a_j^2$: L2 Penalty
 - λ : L2 Regularization의 세기를 조절 하는 하이퍼파라미터
 - 기존의 학습 방식과 같이 Gradient Descent를 통해 모델 파라미터가 학습 됨

Ridge Regression

- Ridge Regression은 모든 파라미터들의 값이 균일하게 작아지도록 학습
- 예시
 - $a_1 = 3, a_2 = 2$ 일 때, Regularization으로 발생하는 loss는 $9 + 4 = 13$
 - Case 1: 학습을 통해 $a_1 = 2, a_2 = 2$ 로 업데이트 $\Rightarrow 4 + 4 = 8$
 - Case 2: 학습을 통해 $a_1 = 3, a_2 = 1$ 로 업데이트 $\Rightarrow 9 + 1 = 10$
 - L2 Penalty를 작게 하려면 큰 값을 갖는 파라미터 값을 줄이는 것이 효율적
- 학습이 진행될 수록 전체 파라미터들이 균일하게 작은 값을 갖게 됨

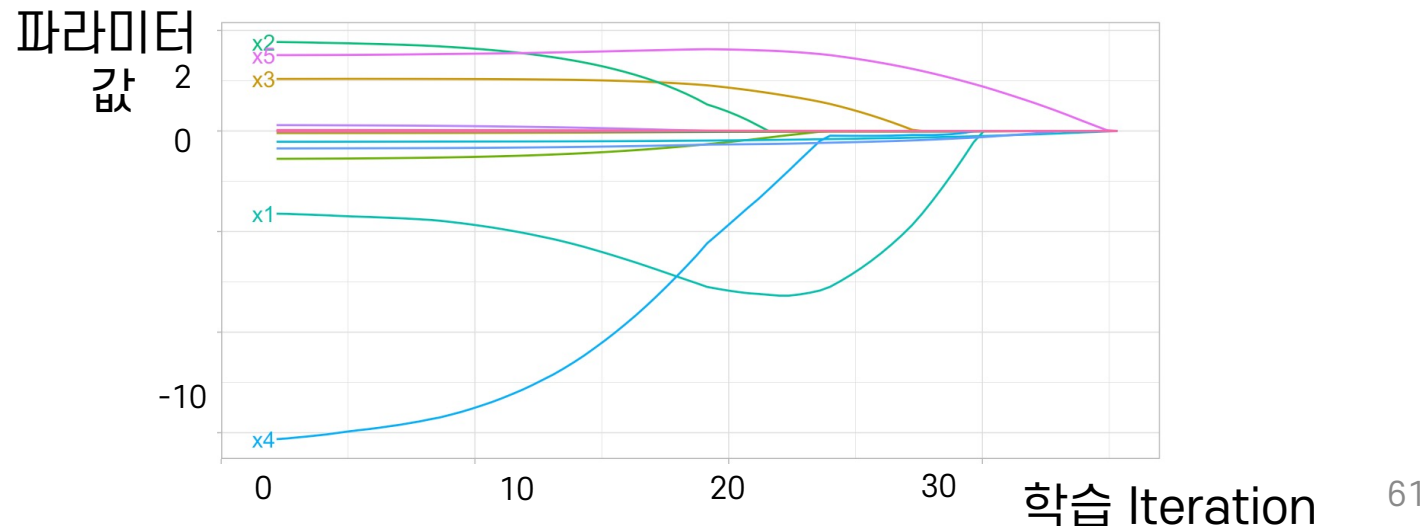


L1 Regularization

- 모델 학습 시 모델 파라미터의 절대 값이 너무 커지지 않도록 제약
- 예시: Lasso Regression
 - 기존 손실 함수 (MSE): $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (y_i - (a_0 + a_1x + a_2x^2 + \dots + a_{10}x^{10}))^2$
 - 의미: MSE를 작게 하는 $a_1, \dots, a_{10}, b$ 를 학습해라
 - 정규화가 추가된 손실 함수: $\frac{1}{N} \sum_{i=1}^N (y_i - (a_0 + a_1x + a_2x^2 + \dots + a_{10}x^{10}))^2 + \lambda \sum_{j=0}^{10} |a_j|$
 - 의미: MSE를 작게 하면서 파라미터의 절대 값이 너무 커지지 않도록 하는 $a_0, a_1, \dots, a_{10}$ 를 학습해라
 - $\sum_{j=0}^{10} |a_j|$: L1 Penalty
 - λ : L1 Regularization의 세기를 조절 하는 하이퍼파라미터
 - 기존의 학습 방식과 같이 Gradient Descent를 통해 모델 파라미터가 학습 됨

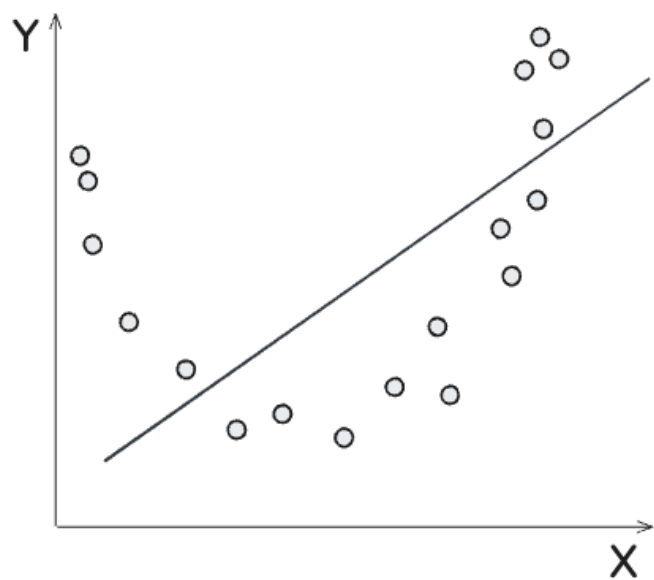
Lasso Regression

- Lasso Regression은 Feature Selection과 같은 효과를 보임
- 예시
 - $a_1 = 3, a_2 = 2$ 일 때, Regularization으로 발생하는 loss는 $3 + 2 = 5$
 - Case 1: 학습을 통해 $a_1 = 2, a_2 = 2$ 로 업데이트 $\Rightarrow 2+2=4$
 - Case 2: 학습을 통해 $a_1 = 3, a_2 = 1$ 로 업데이트 $\Rightarrow 3+1=4$
 - 어떤 파라미터의 값을 줄이더라도 L1 Penalty를 동일하게 줄임
- 학습이 진행될 수록 중요도가 낮은 Feature의 파라미터는 0이 됨

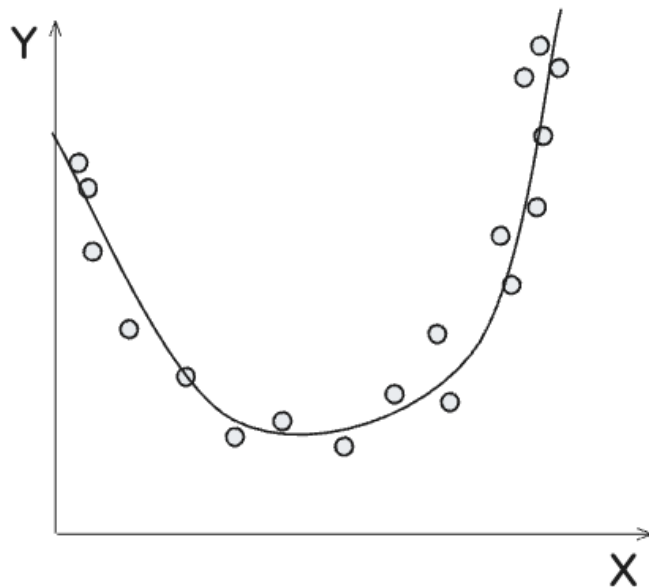


Regularization

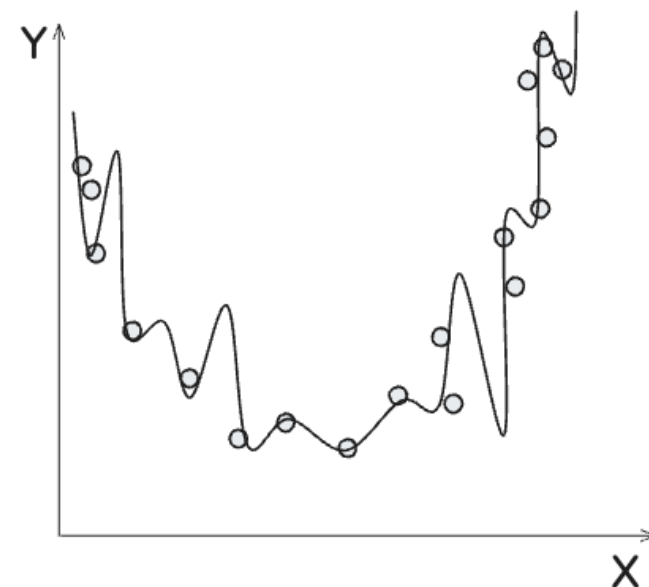
- 하이퍼파라미터 λ 의 영향
 - λ 의 크기에 따라 Loss 값에 대한 Regularization Penalty의 영향력이 결정 됨
 - 최적의 하이퍼파라미터 λ 를 찾는 과정이 필요



너무 큰 λ



적당한 값의 λ



너무 작은 λ

L1, L2 Regularization

- Q5) 머신러닝에서 L1 정규화가 L2 정규화보다 선호되는 경우는 언제인가?
 - L1 정규화는 모델이 결과에 강한 영향을 미치는 희소한(sparse) feature set을 학습할 것으로 예상될 때 선호된다.
 - L1 정규화는 input feature 간 다중공선성(multicollinearity)을 제어해야 할 때 선호된다.
 - L1 정규화는 모델이 overfitting될 가능성이 있고, feature 수를 보존하면서 variance를 줄여야 할 때 선호된다.
 - L1 정규화는 feature 수가 많고 모든 feature가 모델에 고르게 기여하도록 해야 할 때 선호된다.

L1, L2 Regularization

- Q5) 머신러닝에서 L1 정규화가 L2 정규화보다 선호되는 경우는 언제인가?
 - L1 정규화는 모델이 결과에 강한 영향을 미치는 희소한(sparse) feature set을 학습할 것으로 예상될 때 선호된다.
 - L1 정규화는 덜 중요한 feature들의 계수를 0으로 만들어 feature selection을 수행하는 경향이 있음. 이로 인해 희소한 모델이 만들어지며, 결과에 중요한 feature만 남게 되어, 관련 있는 feature만을 모델이 학습하도록 유도할 때 이상적임. 나머지 옵션들은 L2 정규화의 특성에 더 부합하는 조건들임.
 - L1 정규화는 input feature 간 다중공선성(multicollinearity)을 제어해야 할 때 선호된다.
 - L1 정규화는 모델이 overfitting될 가능성이 있고, feature 수를 보존하면서 variance를 줄여야 할 때 선호된다.
 - L1 정규화는 feature 수가 많고 모든 feature가 모델에 고르게 기여하도록 해야 할 때 선호된다.

Code Implementation: Broadcasting

- Q6) 어떤 경우에 대해 Numpy / PyTorch가 오류를 발생시키는가?
 - x와 y가 element-wise로 곱해진다고 가정하자. x와 y의 shape을 각각 s_x 와 s_y 라고 하자.
 - $s_x = [1, 10]$, $s_y = [10, 1]$
 - $s_x = [10, 1]$, $s_y = [10, 1]$
 - $s_x = [100, 10]$ $s_y = [10, 100]$
 - $s_x = [1, 10, 100]$, $s_y = [1, 1, 1]$
- 추가) $[10, 1]$ 이랑 $[4]$ 는 될까 안될까.

Code Implementation: Broadcasting

- Q6) 어떤 경우에 대해 Numpy / PyTorch가 오류를 발생시키는가?
 - x와 y가 element-wise로 곱해진다고 가정하자. x와 y의 shape을 각각 s_x 와 s_y 라고 하자.
 - $s_x = [1, 10]$, $s_y = [10, 1]$
 - $s_x = [10, 1]$, $s_y = [10, 1]$
 - $s_x = [100, 10]$ $s_y = [10, 100]$
 - $s_x = [1, 10, 100]$, $s_y = [1, 1, 1]$
- 추가) $[10, 1]$ 이랑 $[4]$ 는 될까 안될까.
 - 안된다. 그러나, $[1, 10]$ 이랑 $[4]$ 는 된다.
 - $[4]$ 처럼 차원 수가 낮은 shape은 뒤에서부터 채우고 앞은 다 1로 채운다.
E.g., $[1, 4]$

ARIMA Model and Stationarity

- Q7) ARIMA 모델에서 데이터를 차분 (differentiating) 하는 주된 목적은 무엇인가? Discrete한 신호 $x[t]$ 가 있다고 할 때, discrete differentiation (i.e. differencing) 은 아래와 같이 정의된다.

$$y[t] = x[t] - x[t-1]$$

- 모델의 복잡도를 높여 성능을 개선시키기 위해
- 시계열의 정상성 (stationarity)을 확보하기 위해
- 잡음 (noise)을 증가시켜 변화를 잘 보기 위해
- 시계열의 정규화 (normalization)을 위해

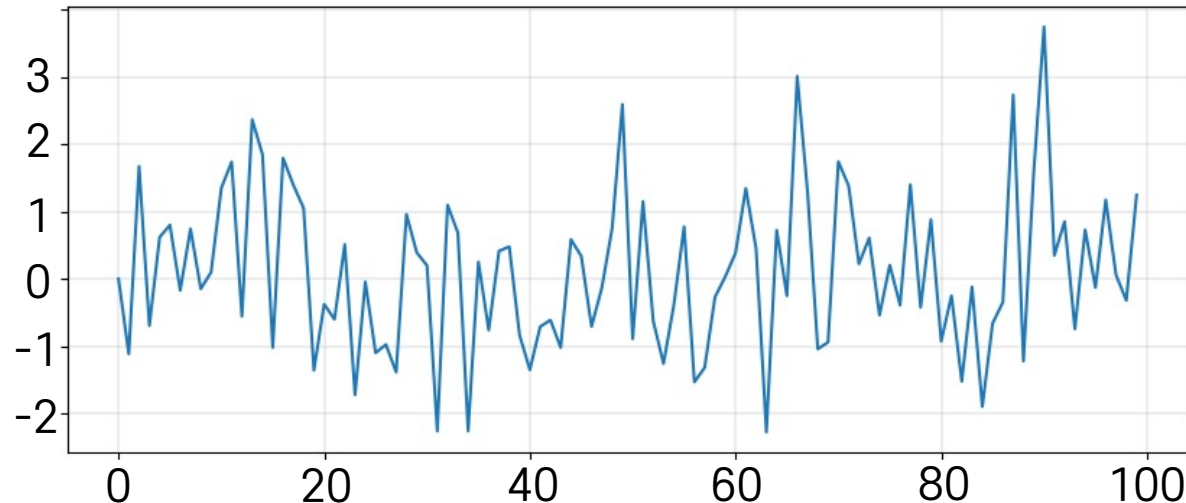
Auto-regressive Process

Auto-regressive (AR) process:

- 현재의 값이 이전 시간 값들의 선형 결합로 표현됨

$$\text{AR}(p): X_t = \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \epsilon_t \quad \text{where } \epsilon_t \sim \text{i.i.d. } N(0, \sigma^2)$$

- p 는 몇 스텝의 과거 값을 사용하는지를 나타내는 Hyperparameter
 - e.g., $\text{AR}(1): X_t = \phi_1 X_{t-1} + \epsilon_t$
- 가까운 시간의 이전 값들을 통해 현재 값을 표현하겠다는 의미
 - e.g., 1시의 주가는 2시의 주가에 영향을 끼침



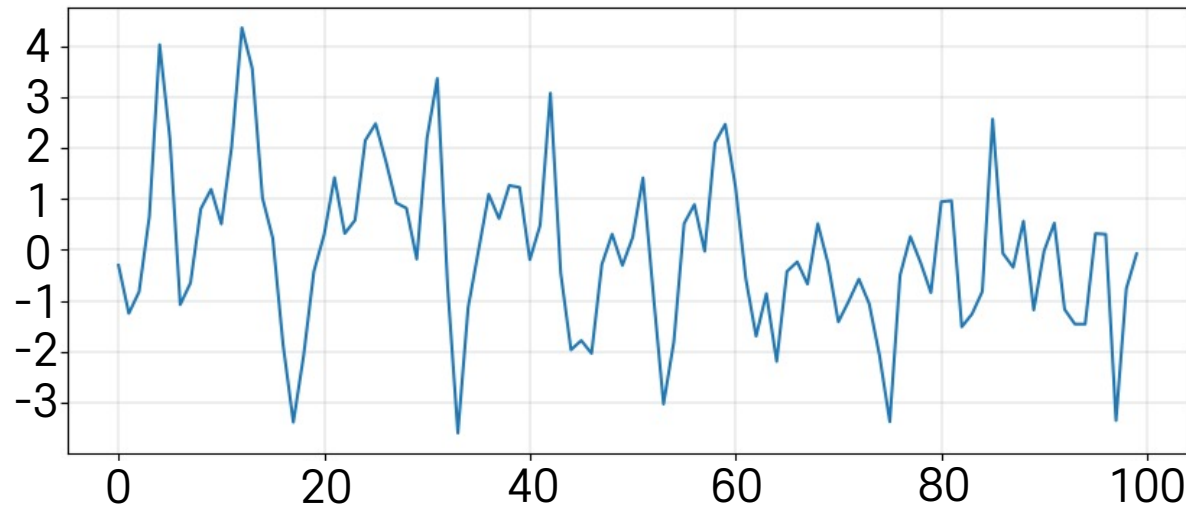
Moving-average process

Moving-average (MA) process

- 현재의 값이 이전 시간 **Noisy error**의 선형 결합에 기반함

$$\text{MA}(q): X_t = \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q} \quad \text{where } \epsilon_t \sim \text{i.i.d. } N(0, \sigma^2)$$

- q 는 몇 스텝의 과거 값을 사용하는지를 나타내는 Hyperparameter
 - e.g., $\text{MA}(1): X_t = \epsilon_t + \theta_1 \epsilon_{t-1}$
- 이전에 발생한 **예측할 수 없는 사건**이 현재까지 영향을 미치는 것을 의미
 - e.g., 1시에 발표한 긴급통화정책이 2시의 주가에 영향을 끼침



ARMA process

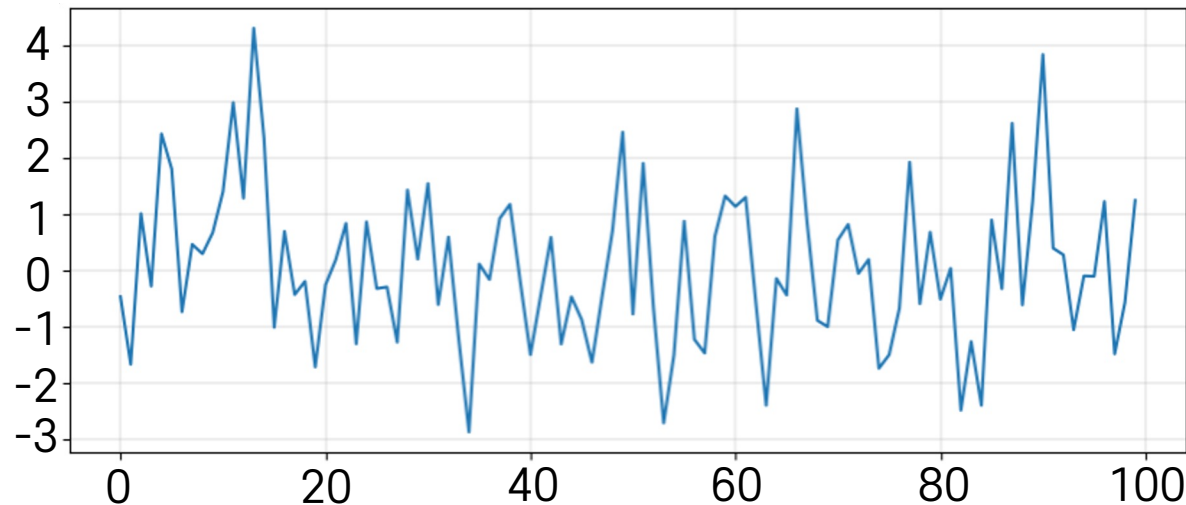
Autoregressive moving average (ARMA) process

- **AR process**와 **MA process**를 함께 사용하여 현재 값을 만드는 Process

$$\text{ARMA}(p, q): X_t = \phi_1 X_{t-1} + \dots + \phi_p X_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$

where $\epsilon_t \sim i.i.d. N(0, \sigma^2)$

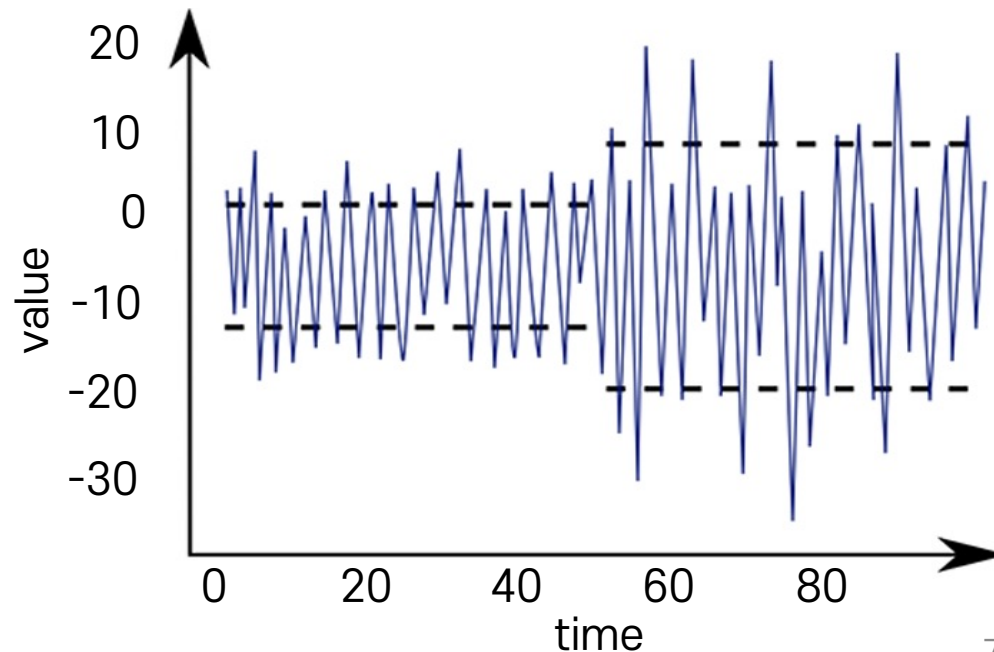
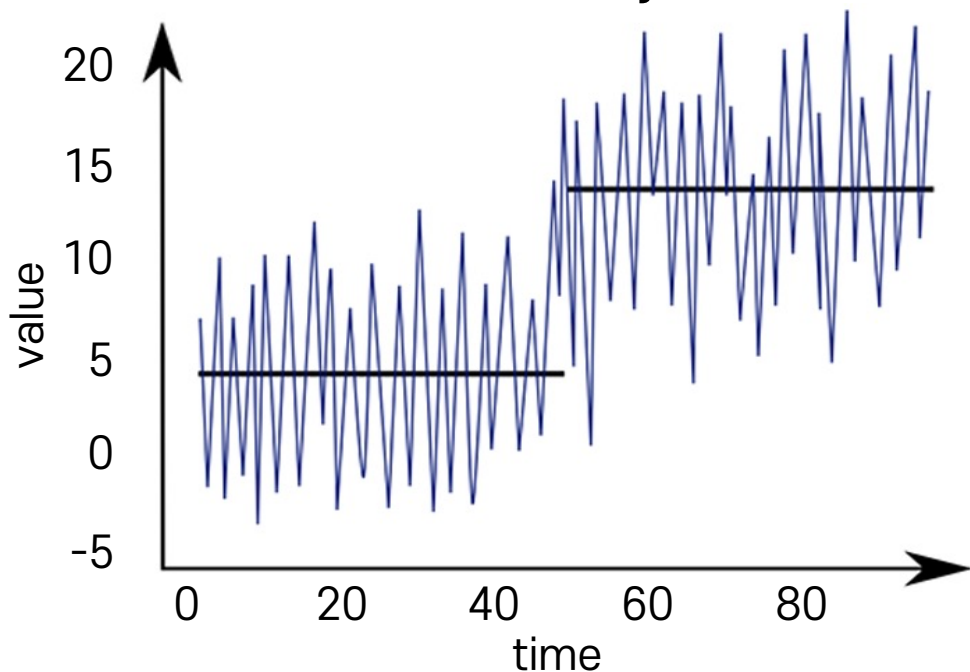
- p, q 는 몇 스텝의 과거 값을 사용하는지를 나타내는 Hyperparameter
 - e.g., $\text{ARMA}(1, 1): X_t = \phi_1 X_{t-1} + \epsilon_t + \theta_1 \epsilon_{t-1}$



Stationarity

시계열 분석에서 데이터와 관련된 가장 중요하고 일반적인 가정

- 하나의 시계열 데이터는 시간에 상관없이, 항상 같은 분포를 따름
- 이것을 만족해야 과거의 관측값을 통해 미래의 관측값을 예측 가능
- Training data와 Test data가 같은 분포에서 나온 것이라는 가정과 같은 의미
- Non-stationary 데이터 시각화 예시



ARIMA Process

Autoregressive integrated moving average (ARIMA) process

- X_t 가 Stationary 하지 않지만 **변화량 ∇X_t 가 Stationary**하다고 가정
- $\nabla^d X_t$ 에 대한 ARMA(p, q) process

ARIMA(p, d, q):

$$\nabla^d X_t = \phi_1 \nabla^d X_{t-1} + \cdots + \phi_p \nabla^d X_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$$

where $\epsilon_t \sim i.i.d. N(0, \sigma^2)$

- p, q 는 몇 스텝의 과거 값을 사용하는지를 나타내는 Hyperparameter
- d 는 Difference operator ∇ 를 몇 번 적용할 것인지에 대한 Hyperparameter
 - $\nabla X_t = X_t - X_{t-1}$
 - $\nabla^2 X_t = (X_t - X_{t-1}) - (X_{t-1} - X_{t-2})$
- e.g., ARIMA(**1**,**1**,**1**): $\nabla X_t = \phi_1 \nabla X_{t-1} + \epsilon_t + \theta_1 \epsilon_{t-1}$

ARIMA Model

ARIMA(p, d, q):

$$\nabla^d X_t = \phi_1 \nabla^d X_{t-1} + \dots + \phi_p \nabla^d X_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$

where $\epsilon_t \sim i.i.d. N(0, \sigma^2)$

ARIMA model을 통한 forecasting은 다음의 네 단계로 요약할 수 있음

1. Hyperparameter p, d, q 를 설정 ($p = 2, d = 1, q = 2$ 가정)
2. $\phi_1, \phi_2, \theta_1, \theta_2$ 를 주어진 데이터의 에러를 최소화 하도록 학습
 - MSE (Mean Squared Error) 또는 AIC (Akaike's Information Criterion) 등을 최소화
 - 1) AR(2) 모델을 주어진 데이터 ∇X 로 학습
 - 2) 학습된 AR(2) 모델로 ∇X 의 예측값 $\nabla X'$ 을 계산
 - 3) 예측 오차 $\epsilon = \nabla X - \nabla X'$ 을 계산
 - 4) 계산된 예측 오차 ϵ 를 데이터로 사용하여 MA(2) 모델을 학습

ARIMA Model

ARIMA(p, d, q):

$$\nabla^d X_t = \phi_1 \nabla^d X_{t-1} + \dots + \phi_p \nabla^d X_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$

where $\epsilon_t \sim i.i.d. N(0, \sigma^2)$

ARIMA model을 통한 forecasting은 다음의 네 단계로 요약할 수 있음

3. $\nabla X_{t-1}, \nabla X_{t-2}, \epsilon_{t-1}, \epsilon_{t-2}$ 를 대입하여 t 시점의 ∇X_t 를 예측

1) $\nabla X_{t-1}, \nabla X_{t-2}$ 과 AR(2) 모델을 이용하여 $\nabla X'_t$ 을 예측

2) $\epsilon_{t-1}, \epsilon_{t-2}$ 와 MA(2) 모델을 이용하여 ϵ'_t 을 예측

3) 두 예측값 $\nabla X'_t$ 와 ϵ'_t 을 더하여 ∇X_t 를 계산

4. $X_t = X_{t-1} + \nabla X_t$ 로 X_t 의 예측값을 얻을 수 있음

ARIMA Model and Stationarity

- Q7) ARIMA 모델에서 데이터를 차분 (differentiating) 하는 주된 목적은 무엇인가? Discrete한 신호 $x[t]$ 가 있다고 할 때, discrete differentiation (i.e. differencing) 은 아래와 같이 정의된다.

$$y[t] = x[t] - x[t-1]$$

- 모델의 복잡도를 높여 성능을 개선시키기 위해
- 시계열의 정상성 (stationarity)을 확보하기 위해
- 잡음 (noise)을 증가시켜 변화를 잘 보기 위해
- 시계열의 정규화 (normalization)을 위해

ARIMA Model and Stationarity

- Q7) ARIMA 모델에서 데이터를 차분 (differentiating) 하는 주된 목적은 무엇인가? Discrete한 신호 $x[t]$ 가 있다고 할 때, discrete differentiation (i.e. differencing) 은 아래와 같이 정의된다.

$$y[t] = x[t] - x[t-1]$$

- 모델의 복잡도를 높여 성능을 개선시키기 위해
- 시계열의 정상성 (stationarity)을 확보하기 위해
- 잡음 (noise)을 증가시켜 변화를 잘 보기 위해
- 시계열의 정규화 (normalization)을 위해

Autocovariance and Autocorrelation

- Q8) 다음 중 Autocorrelation function (ACF) 그래프에 대한 해석으로 옳은 것은?
 - ACF는 autocovariance를 정규화하여 구할 수 있다.
 - ACF가 천천히 감소하면, 데이터의 랜덤성이 매우 높다.
 - ACF는 항상 양의 값을 가진다.
 - 정상성 (stationarity)이 높은 시계열은 항상 ACF 값이 높게 유지된다.

Autocovariance

Autocovariance: 하나의 시계열 데이터의 내부적인 연관성을 표현하는 수치

$$\begin{aligned}\gamma(\tau) &= \gamma_{XX}(\tau) = \text{Cov}[X_t, X_{t-\tau}] \\ &= \mathbb{E}[(X_t - \bar{X}_t)(X_{t-\tau} - \bar{X}_{t-\tau})]\end{aligned}$$

- 하나의 시계열 데이터와 그것을 τ 만큼 이동시킨 시계열 데이터의 **Covariance**
- 데이터를 통한 Stationary process X_t 의 $\gamma(\tau)$ 를 추정

$$\hat{\gamma}(\tau) = \frac{1}{N} \sum_{t=1}^{N-\tau} (X_t - \bar{X}_t)(X_{t+\tau} - \bar{X}_{t+\tau}) = \frac{1}{N} \sum_{t=1}^{N-\tau} (X_t - \bar{X})(X_{t+\tau} - \bar{X})$$

$$\text{where } \bar{X} = \frac{1}{N} \sum_{t=1}^N X_t$$

Autocovariance

Stationary process에서 Autocovariance 계산 예시

- $X = [1, 3, 5, 7, 9]$
 - 평균 $\bar{X} = 5$, 분산 $\hat{\sigma}_X^2 = \frac{\sum_{t=1}^N (X_t - \bar{X})^2}{N} = 8$
- Stationary process의 특징
 - 모든 t 에 대해서 $\bar{X}_t = \bar{X} = 5$, $\hat{\sigma}_t^2 = \hat{\sigma}_X^2 = 8$
- Autocovariance $\gamma_{XX}(\tau = 1)$
 - $X = [1, 3, 5, 7, 9]$, $X^{(\tau)} = [3, 5, 7, 9, \text{None}]$
 - $\frac{1}{5}((1 - 5)(3 - 5) + (3 - 5)(5 - 5) + (5 - 5)(7 - 5) + (7 - 5)(9 - 5)) = \frac{1}{5}(8 + 0 + 0 + 8) = \frac{16}{5}$

Autocovariance

$$\hat{\gamma}(\tau) = \frac{1}{N} \sum_{t=1}^{N-\tau} (X_t - \bar{X}_t)(X_{t+\tau} - \bar{X}_{t+\tau})$$

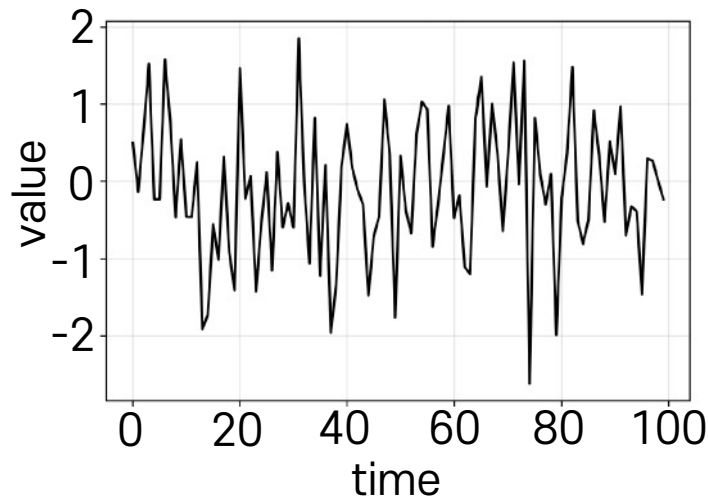
Autocorrelation

Autocorrelation: 분산 값을 이용하여 Autocovariance의 값을 정규화한 값

$$\rho(\tau) = \rho_{XX}(\tau) = \frac{\gamma_{XX}(\tau)}{\sigma_X \sigma_{X-\tau}} = \frac{\text{Cov}[X(t), X(t - \tau)]}{\sigma_X \sigma_{X-\tau}}$$

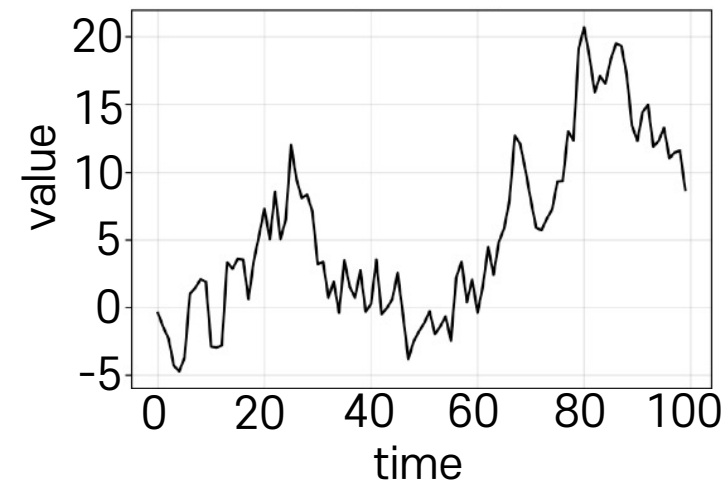
- -1과 +1 사이의 값을 가짐
- Stationary process X_t 의 $\rho(\tau)$ 의 추정: $\hat{\rho}(\tau) = \frac{\hat{\gamma}(\tau)}{\hat{\sigma}_X \hat{\sigma}_{X-\tau}} = \frac{\hat{\gamma}(\tau)}{\hat{\sigma}_X^2}$

Autocorrelation 존재하지 않음



특정 시간에서의
상승 하락이
다른 시간의
상승 하락과
전혀 상관성이 없음

Autocorrelation 존재



Time 25 부근의
상승 하락과
Time 80 부근의
상승 하락이
상관성을 갖고 있음

Autocorrelation

Stationary process에서 Autocorrelation 예시

- $X = [1, 3, 5, 7, 9]$
 - 평균 $\bar{X} = 5$, 분산 $\hat{\sigma}_X^2 = 8$
- Stationary process의 특징
 - 모든 t 에 대해서 $\bar{X}_t = \bar{X} = 5$, $\hat{\sigma}_t^2 = \hat{\sigma}_X^2 = 8$
- Autocovariance $\gamma_{XX}(\tau = 1)$
 - $X = [1, 3, 5, 7, 9]$, $X^{(\tau)} = [3, 5, 7, 9, \text{None}]$
 - $\frac{1}{5}((1 - 5)(3 - 5) + (3 - 5)(5 - 5) + (5 - 5)(7 - 5) + (7 - 5)(9 - 5)) = \frac{1}{5}(8 + 0 + 0 + 8) = \frac{16}{5}$
- Autocorrelation $\rho(1)$
 - $\frac{16}{5} \times \frac{1}{8} = \frac{2}{5}$

Autocovariance

$$\hat{\gamma}(\tau) = \frac{1}{N} \sum_{t=1}^{N-\tau} (X_t - \bar{X}_t)(X_{t+\tau} - \bar{X}_{t+\tau})$$

Autocorrelation

$$\rho_{XX}(\tau) = \frac{\gamma_{XX}(\tau)}{\sigma_X \sigma_{X-\tau}} = \frac{\gamma_{XX}(\tau)}{\sigma_X \sigma_X} = \frac{\gamma_{XX}(\tau)}{\gamma_{XX}(0)}$$

Autocovariance and Autocorrelation

- Q8) 다음 중 Autocorrelation function (ACF) 그래프에 대한 해석으로 옳은 것은?
 - ACF는 autocovariance를 정규화하여 구할 수 있다.
 - ACF가 천천히 감소하면, 데이터의 랜덤성이 매우 높다.
 - ACF는 항상 양의 값을 가진다.
 - 정상성 (stationarity)이 높은 시계열은 항상 ACF 값이 높고 유지된다.

Autocovariance and Autocorrelation

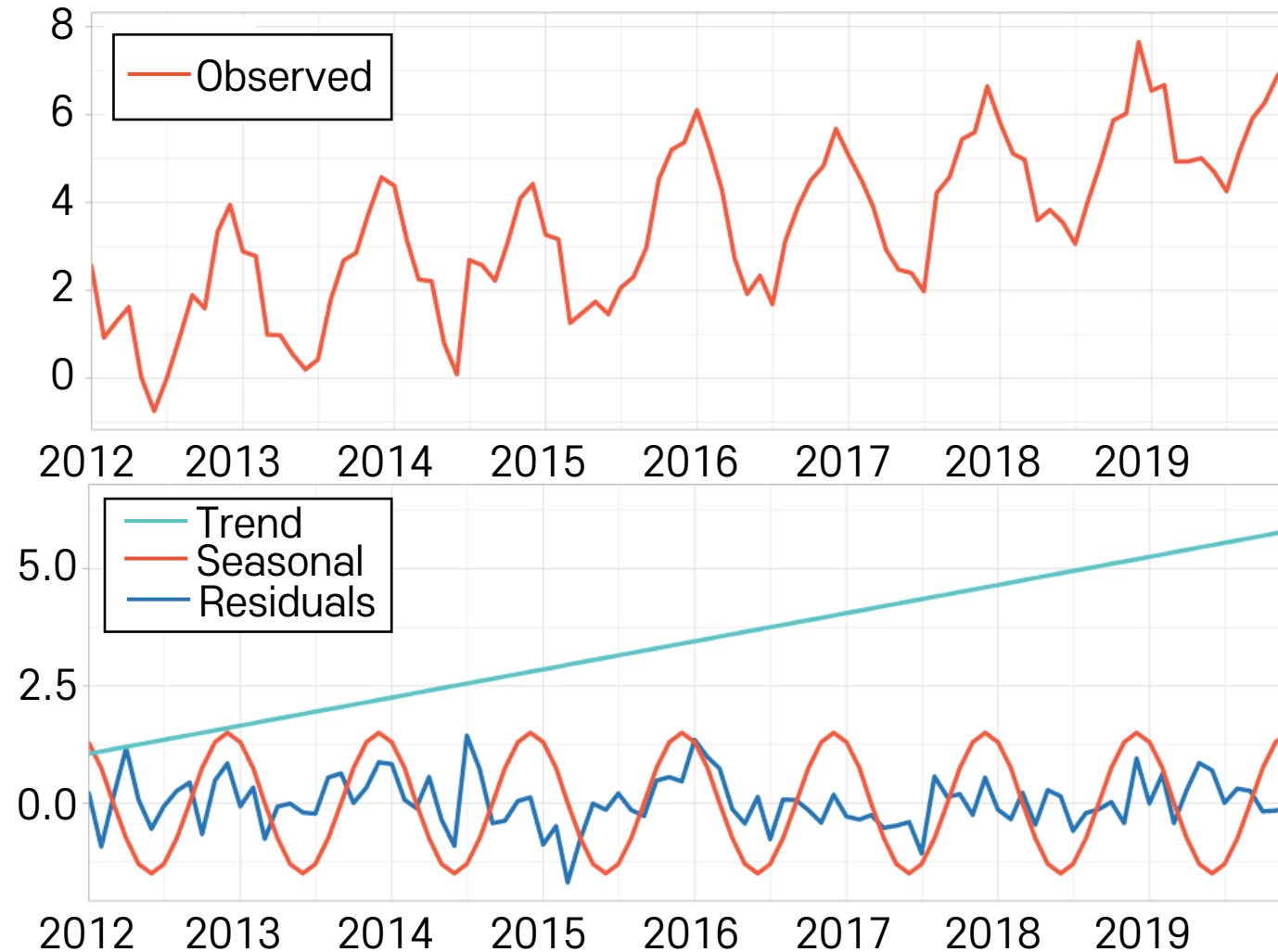
- Q8) 다음 중 Autocorrelation function (ACF) 그래프에 대한 해석으로 옳은 것은?
 - ACF는 autocovariance를 정규화하여 구할 수 있다.
 - ACF가 천천히 감소하면, 데이터의 랜덤성이 매우 높다.
 - 랜덤 데이터는 lag 0 이후에 ACF 값이 거의 0에 가깝게 떨어짐.
 - ACF는 항상 양의 값을 가진다.
 - 음의 값도 가질 수 있다.
 - 정상성 (stationarity)이 높은 시계열은 항상 ACF 값이 높고 유지된다.
 - 오히려 ACF 값이 빠르게 감소한다.

Time Series Decomposition

- Q9) 시계열 분해를 additive하게 할 때 가장 옳은 설명은?
 - 시계열의 계절적 (seasonal) 변화의 크기가 trend의 크기에 비례하여 변한다.
 - Trend와 seasonality 성분이 서로 독립적이며, 시간이 지남에 따라 변화의 진폭이 일정하다.
 - Trend가 존재하지 않을 때만 additive 분해를 할 수 있다.
 - Residual이 항상 정상적 stationary하지 않아도 상관 없다.

Time Series Decomposition

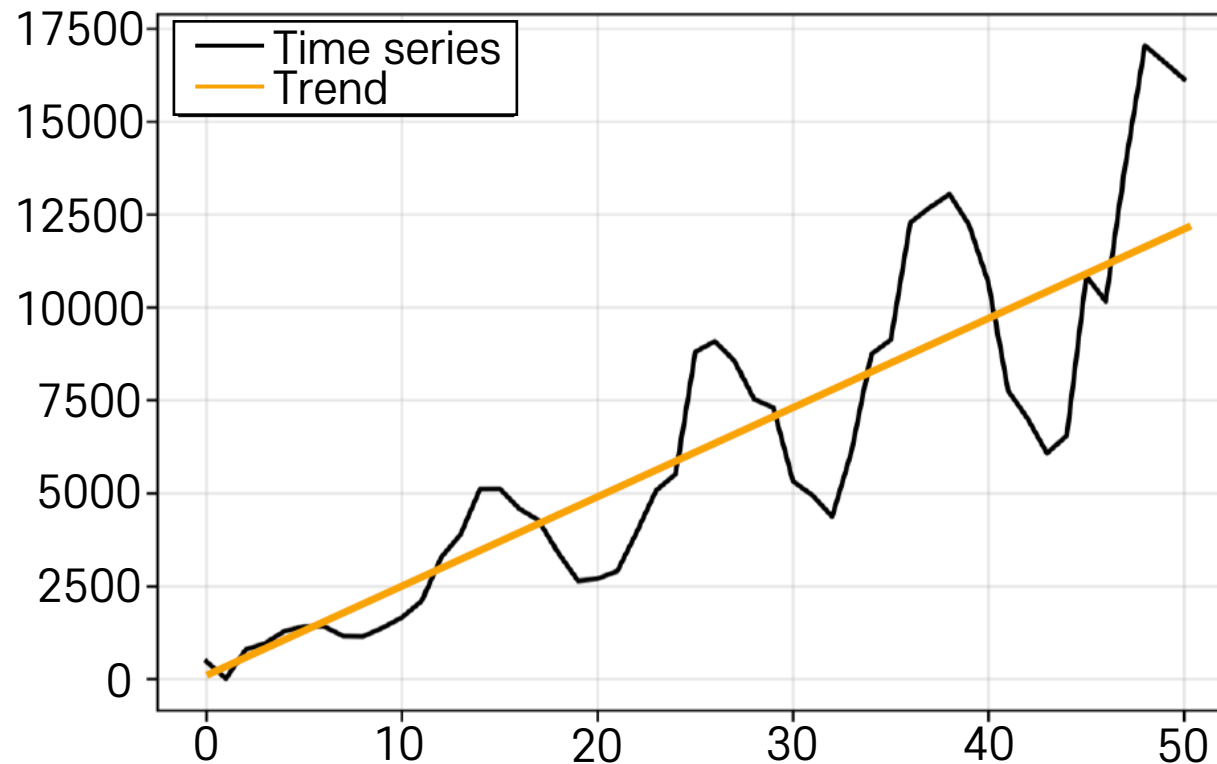
- 시계열 데이터는 **Trend**, **Seasonality**, **Residuals**로 분해 가능



Trend

Trend

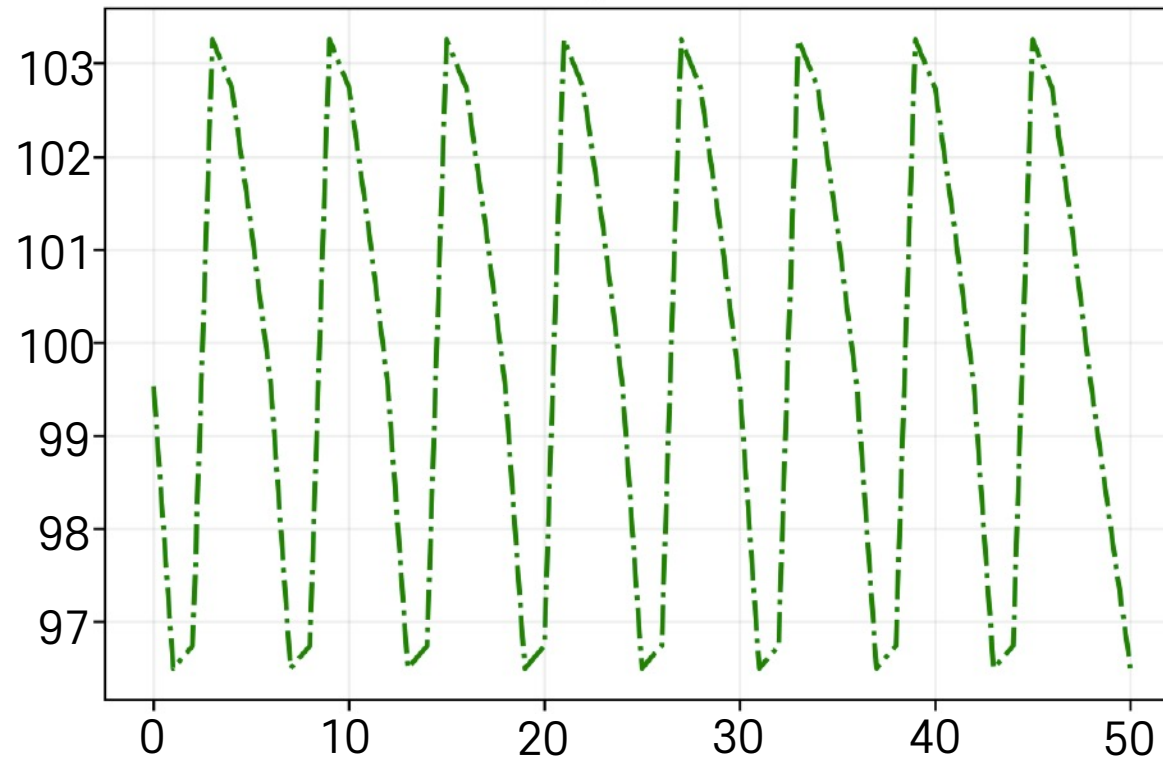
- 시계열 데이터의 증가와 감소 같은 전체적인 경향성을 표현
- 랜덤한 잡음이 없는 결정적인 (Deterministic) 요소를 표현



Seasonality

Seasonality

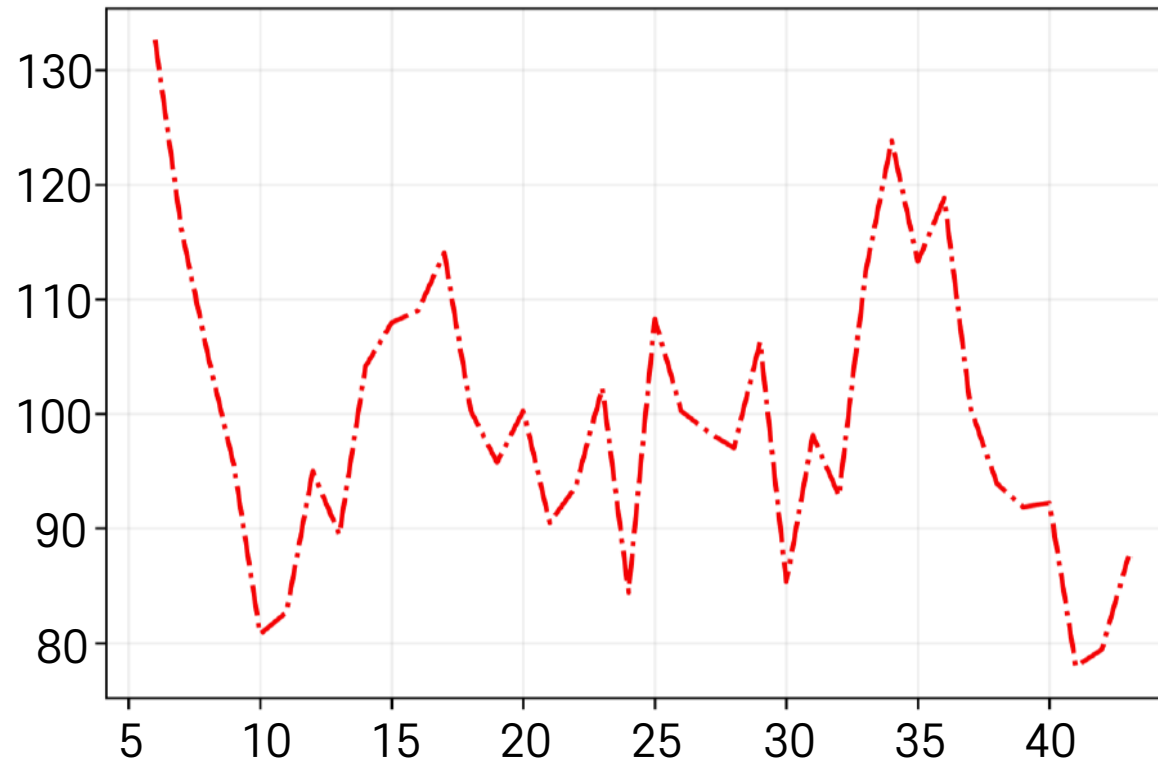
- 시계열 데이터의 주기성을 표현
- 주기성은 여러가지 요인에 의해 발생할 수 있음
 - 시간, 요일, 월, 연도, 계절



Residuals

Residuals

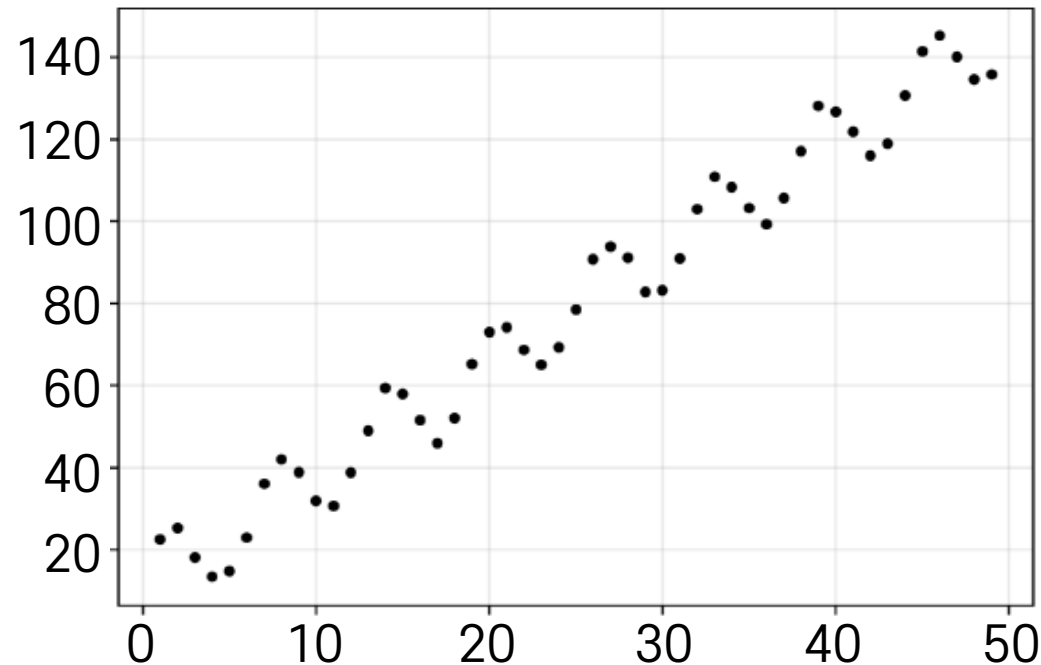
- **Trend**와 **Seasonality**를 제거한 후에 남아있는 랜덤한 요소
- Trend와 Seasonality 요소를 완전히 제거하지 못한다면,
Residuals에 여전히 특정 경향성이나 주기성이 나타날 수 있음



Time Series Decomposition

시계열의 세가지 요소는 합 또는 곱의 형태로 결합 되어 있음

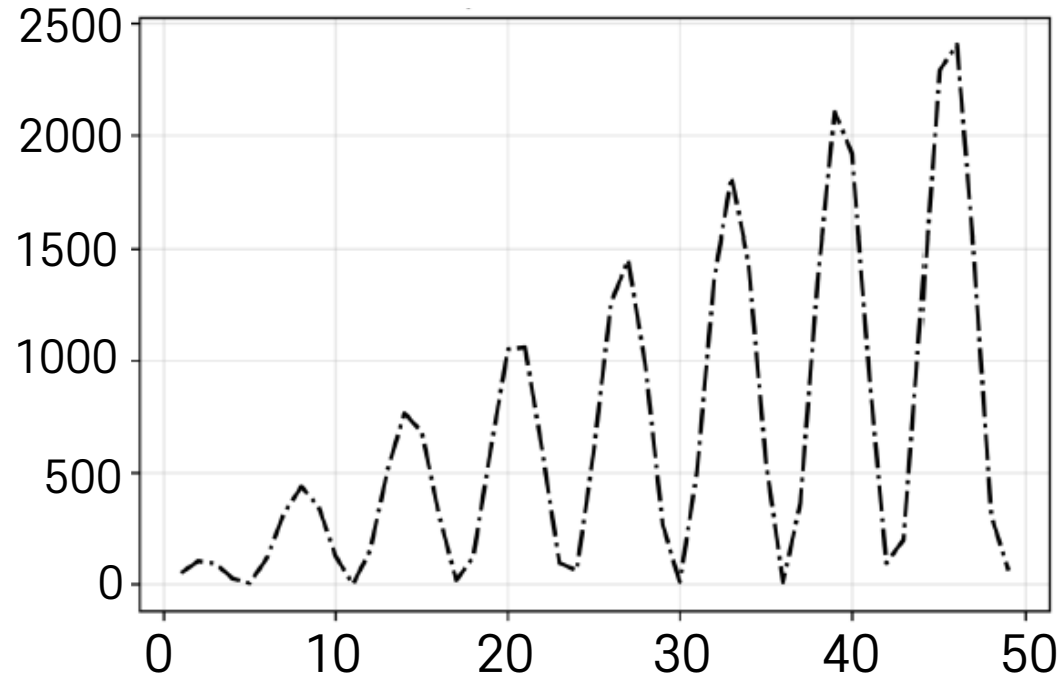
- Case 1) $\text{Data} = \text{Trend} + \text{Seasonality} + \text{Residual}$
 - Trend의 방향을 따라서 시간에 따른 변화폭이 일정함



Time Series Decomposition

시계열의 세가지 요소는 합 또는 곱의 형태로 결합 되어 있음

- Case 2) $\text{Data} = \text{Trend} \times \text{Seasonality} \times \text{Residual}$
 - Trend의 방향과 상관없이 시간에 따른 변화폭이 일정하지 않음



- 곱의 형태는 log 연산을 통해 합의 형태로 변환할 수 있음
 - $\log(\text{Data}) = \log(\text{Trend}) + \log(\text{Seasonality}) + \log(\text{Residual})$

Time Series Decomposition

- Q9) 시계열 분해를 additive하게 할 때 가장 옳은 설명은?
 - 시계열의 계절적 (seasonal) 변화의 크기가 trend의 크기에 비례하여 변한다.
 - Trend와 seasonality 성분이 서로 독립적이며, 시간이 지남에 따라 변화의 진폭이 일정하다.
 - Trend가 존재하지 않을 때만 additive 분해를 할 수 있다.
 - Residual이 항상 정상적 stationary하지 않아도 상관 없다.

Time Series Decomposition

- Q9) 시계열 분해를 additive하게 할 때 가장 옳은 설명은?
 - 시계열의 계절적 (seasonal) 변화의 크기가 trend의 크기에 비례하여 변한다.
 - multiplicative decomposition에 대한 설명이다.
 - Trend와 seasonality 성분이 서로 독립적이며, 시간이 지남에 따라 변화의 진폭이 일정하다.
 - Trend가 존재하지 않을 때만 additive 분해를 할 수 있다.
 - 오히려 trend를 명확히 분리하기 위해 사용된다.
 - Residual이 항상 정상적 stationary하지 않아도 상관 없다.
 - decomposition했을 때의 목표 중의 하나는 residual을 stationary하게 만드는 것이다.

Code Implementation: Sparsity measurement

- Q10) Sparsity measurement의 올바른 구현은?

- $$M(q_i, K) = \max_j \left\{ \frac{(q_i k_j^T)}{\sqrt{d}} \right\} - \frac{1}{L_K} \sum_{j=1}^{L_K} \frac{q_i k_j^T}{\sqrt{d}}$$

- Sparsity measurement는 위와 같이 정의된다. 여기서 모든 dot product의 계산을 방지하기 위해, u 만큼의 dot product만 random sampling하여 계산하는 방식을 취한다.

```
1 sampled_dot_products = np.prod(sampled_keys, q)
2 M_value = np.max(sampled_dot_products) - np.mean(sampled_dot_products)
```

```
1 sampled_dot_products = np.prod(sampled_keys, q) / scale
2 M_value = np.max(sampled_dot_products) - np.mean(sampled_dot_products)
```

```
1 sampled_dot_products = np.dot(sampled_keys, q) / scale
2 M_value = np.max(sampled_dot_products) - np.sum(np.mean(sampled_dot_products))
```

```
1 sampled_dot_products = np.dot(sampled_keys, q) / scale
2 M_value = np.max(sampled_dot_products) - np.mean(sampled_dot_products)
```

Code Implementation: Sparsity measurement

- Q10) Sparsity measurement의 올바른 구현은?

- $$M(q_i, K) = \max_j \left\{ \frac{(q_i k_j^T)}{\sqrt{d}} \right\} - \frac{1}{L_K} \sum_{j=1}^{L_K} \frac{q_i k_j^T}{\sqrt{d}}$$

- Sparsity measurement는 위와 같이 정의된다. 여기서 모든 dot product의 계산을 방지하기 위해, u 만큼의 dot product만 random sampling하여 계산하는 방식을 취한다.

```
1 sampled_dot_products = np.prod(sampled_keys, q)
2 M_value = np.max(sampled_dot_products) - np.mean(sampled_dot_products)
```

```
1 sampled_dot_products = np.prod(sampled_keys, q) / scale
2 M_value = np.max(sampled_dot_products) - np.mean(sampled_dot_products)
```

```
1 sampled_dot_products = np.dot(sampled_keys, q) / scale
2 M_value = np.max(sampled_dot_products) - np.sum(np.mean(sampled_dot_products))
```

```
1 sampled_dot_products = np.dot(sampled_keys, q) / scale
2 M_value = np.max(sampled_dot_products) - np.mean(sampled_dot_products)
```

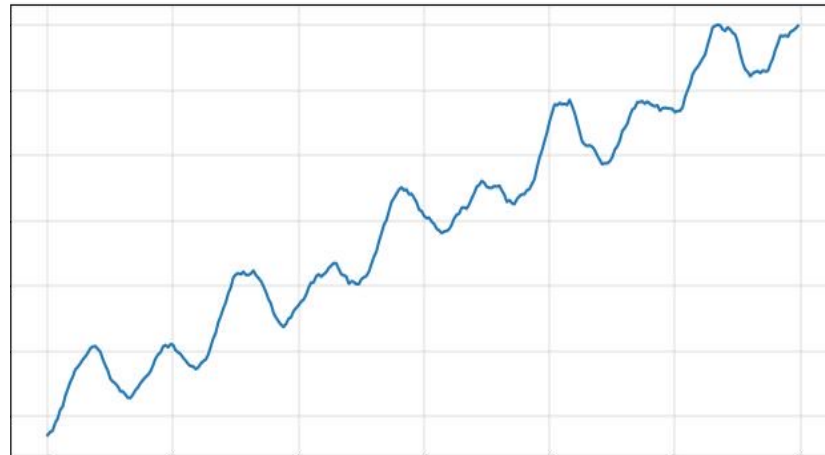
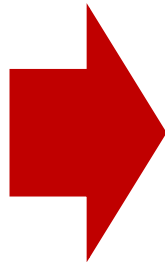
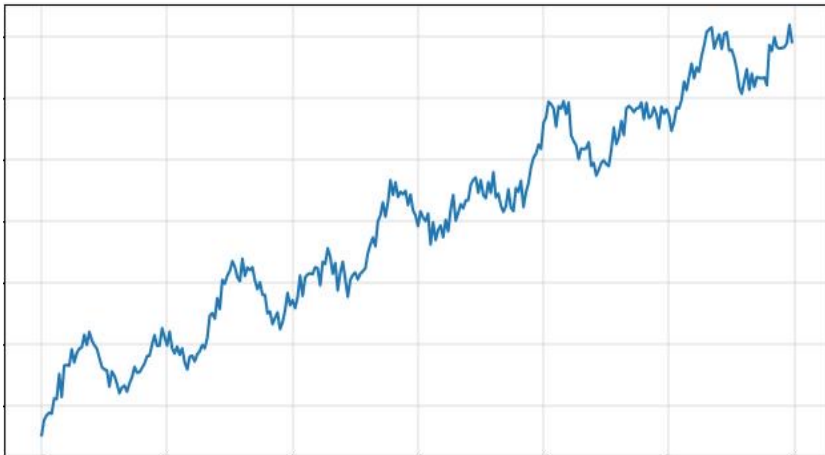
Data Smoothing

- Q11) 시계열 데이터를 예측할 때, Exponential Moving Average (EMA) 기법의 특징으로 옳은 것은?
 - 최근 데이터보다는 과거 데이터를 더 높은 가중치로 반영한다.
 - 단기 예측보다는 장기 예측에 적합하다.
 - 단순 이동 평균보다 변화에 빠르게 반응할 수 있다.
 - 항상 계절성을 고려하여 예측한다.

Data Smoothing

시계열 데이터를 각 요소별로 분해할 수 있는 방법

- Data smoothing
 - Moving average
 - Exponential moving average
- Data smoothing을 통해 trend 추출과 잡음(또는 residual) 제거 가능



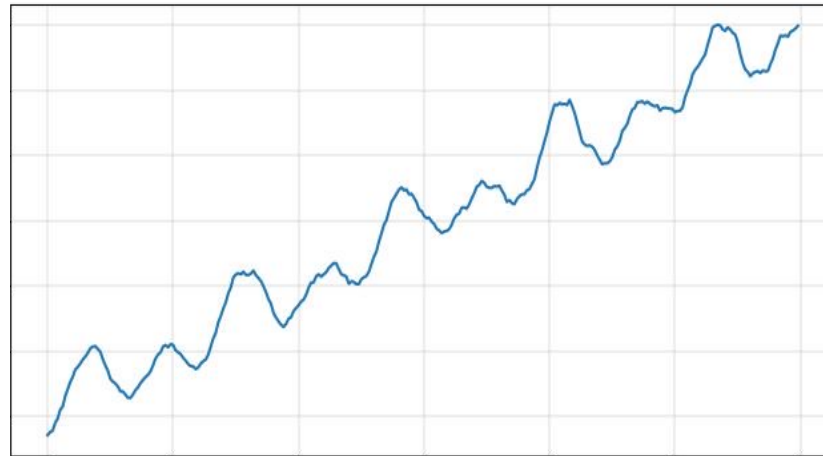
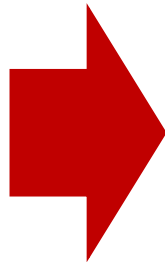
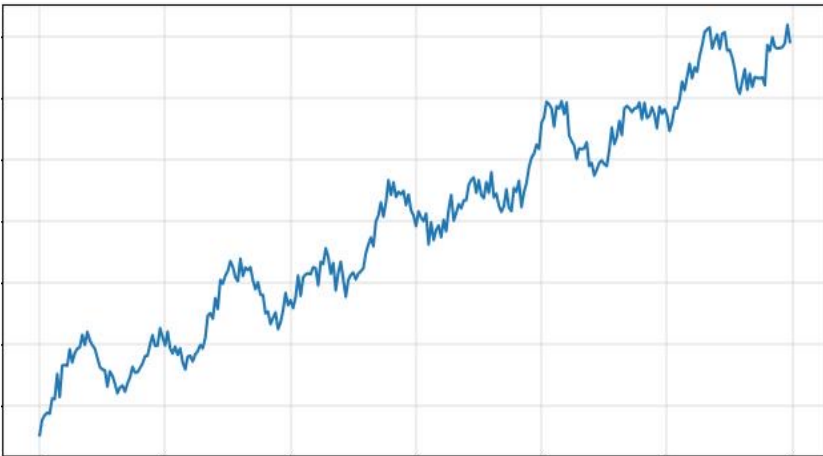
Data Smoothing

Moving average

- 가장 간단한 Data smoothing 방식
- 한 값을 기준으로 정해놓은 크기의 범위만큼 양 옆의 값들을 평균을 취함

$$X'_t = \frac{X_{t-2} + X_{t-1} + X_t + X_{t+1} + X_{t+2}}{5}$$

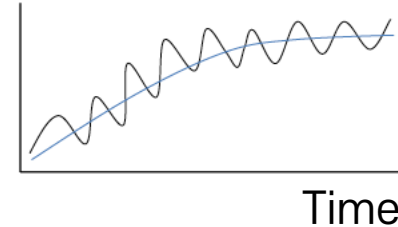
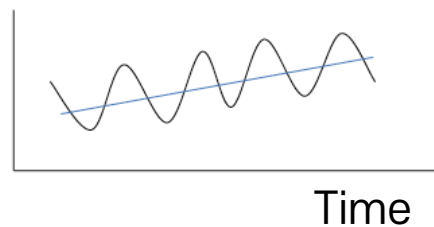
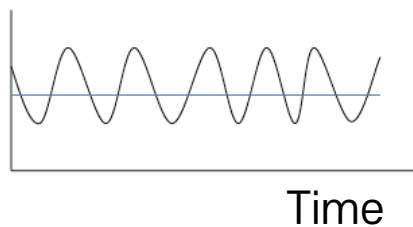
- 시계열 데이터에서 잡음을 제거할 때 사용 가능



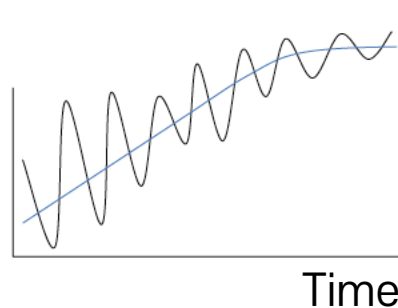
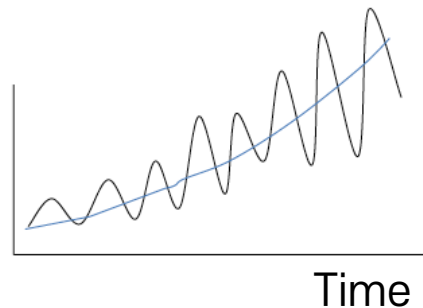
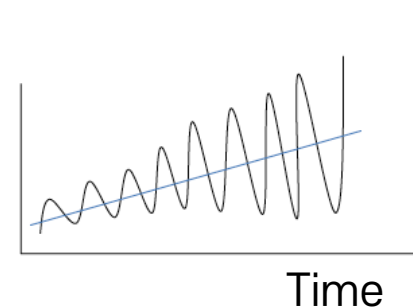
Data Smoothing

Exponential moving average

- 모든 값을 같은 중요도로 평균을 취하는 것이 아니라,
가까운 시간의 관측치일수록 중요도를 높게 배정하여 평균을 취함
- $0 < w < 1$
- $X'_t = w^3 X_{t-2} + w^2 X_{t-1} + w X_t + w^2 X_{t+1} + w^3 X_{t+2}$
- 데이터에서 Trend를 추출하는 것에 효과적



— 원본 데이터
— Smoothing 결과



Data Smoothing

- Q11) 시계열 데이터를 예측할 때, Exponential Moving Average (EMA) 기법의 특징으로 옳은 것은?
 - 최근 데이터보다는 과거 데이터를 더 높은 가중치로 반영한다.
 - 단기 예측보다는 장기 예측에 적합하다.
 - 단순 이동 평균보다 변화에 빠르게 반응할 수 있다.
 - 항상 계절성을 고려하여 예측한다.

Data Smoothing

- Q11) 시계열 데이터를 예측할 때, Exponential Moving Average (EMA) 기법의 특징으로 옳은 것은?
 - 최근 데이터보다는 과거 데이터를 더 높은 가중치로 반영한다.
 - EMA는 최근 데이터에 더 높은 가중치를 부여한다.
 - 단기 예측보다는 장기 예측에 적합하다.
 - 단기 예측에 유리한 방법이다.
 - 단순 이동 평균보다 변화에 빠르게 반응할 수 있다.
 - 이동 평균보다 가중치를 지수적으로 감소시키므로 변화에 더 빠르게 반응한다.
 - 항상 계절성을 고려하여 예측한다.
 - 기본적인 E 기법은 계절성을 고려하지 않는다. 계절성을 포함하려면 Holt-Winters 모델을 사용해야 한다.

Regularization

- Q12) 타임시리즈 예측을 위해 LSTM 기반 모델을 학습하던 중, validation loss가 계속 증가하는 현상이 발생하였다. 아래 중 이 문제를 해결하는 데 가장 효과적인 방법은 무엇인가?
 - 더 많은 과거 시점을 입력으로 사용한다.
 - Dropout을 추가하거나 비율을 증가시킨다.
 - 시계열 데이터에 대해 정규화를 적용한다.
 - L2 Weight regularization을 걸어준다.

Regularization

- Q12) 타임시리즈 예측을 위해 LSTM 기반 모델을 학습하던 중, validation loss가 계속 증가하는 현상이 발생하였다. 아래 중 이 문제를 해결하는 데 가장 효과적인 방법은 무엇인가?
 - 더 많은 과거 시점을 입력으로 사용한다.
 - 더 많은 과거 입력을 사용하면 정보는 늘어나지만, 오히려 오버피팅이 심해질 수 있고, validation loss를 줄이는 데는 도움이 안 될 수 있음. 특히 LSTM은 긴 시퀀스에 대해 gradient vanishing 문제가 발생할 수도 있음.
 - Dropout을 추가하거나 비율을 증가시킨다.
 - 시계열 데이터에 대해 정규화를 적용한다.
 - L2 Weight regularization을 걸어준다.